Aula 07 – Caminho de dados e controle MIPS Pipeline

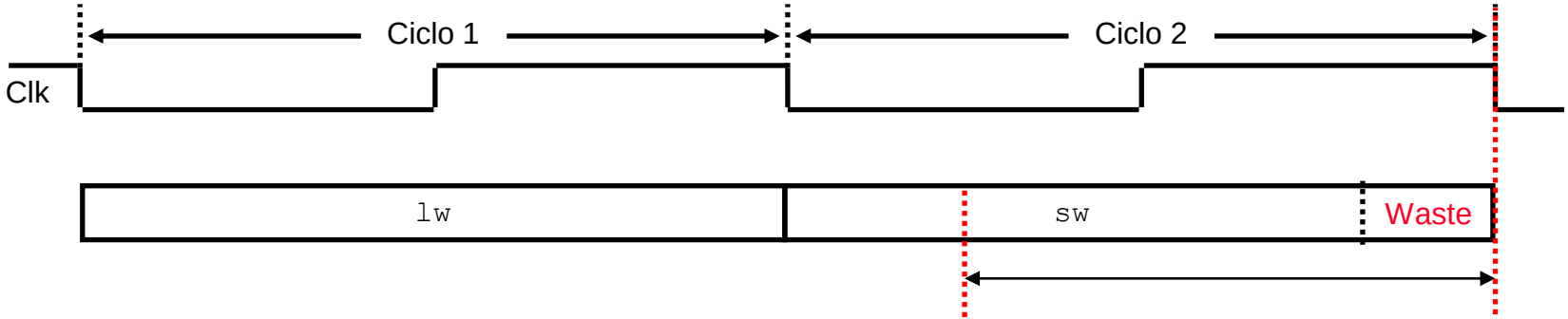
Prof. João Fernando Mari
joaofmari.github.io

Roteiro

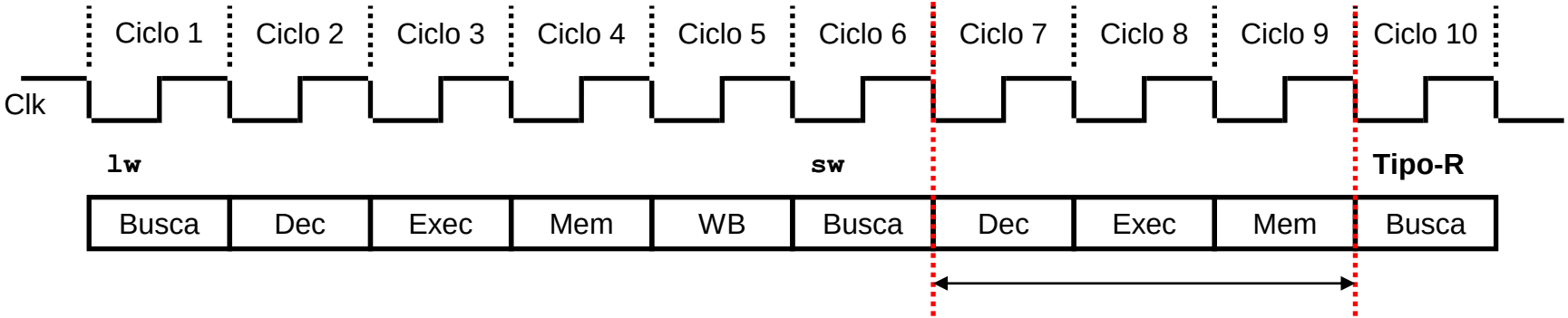
- Desempenho: Monociclo vs. Multiciclo
- Pipelining – Definição
- Desempenho: Monociclo vs Multiciclo vs. Pipelining
- Pipelining Hazards
  - Hazards Estruturais (Restrições físicas do hardware)
  - Hazards de Dados (Dados incompletos)
  - Hazards de Controle (Saltos e desvios)
- MIPS Pipeline – Caminho de dados e controle
  - Divisão de instrução em cinco estágios
  - Instrução load word (lw)
  - Instrução store word (sw)
  - Controle de um pipeline

Monociclo Vs. Multiciclo

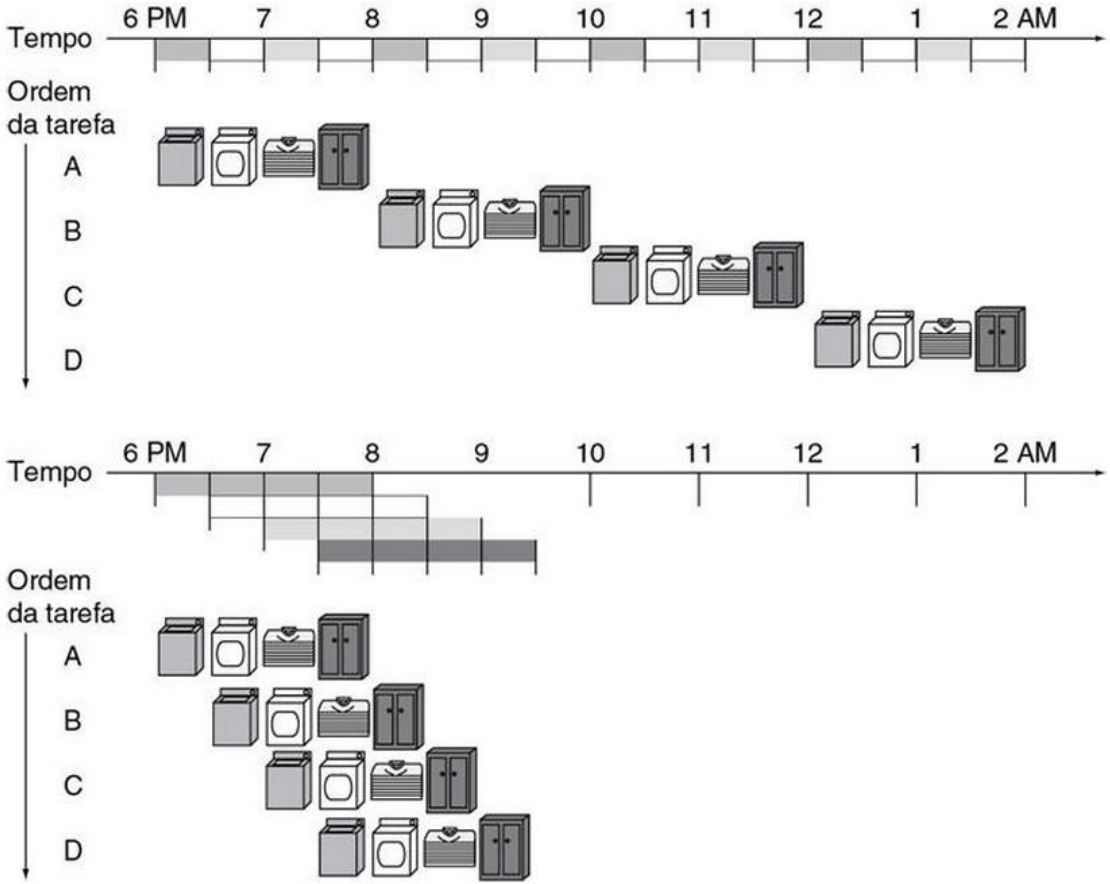
Implementação de ciclo único:



Implementação multiciclo:

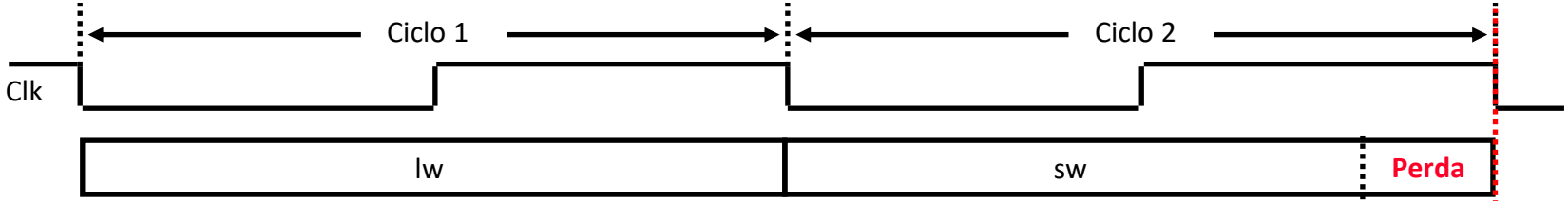


Pipelining (analogia)

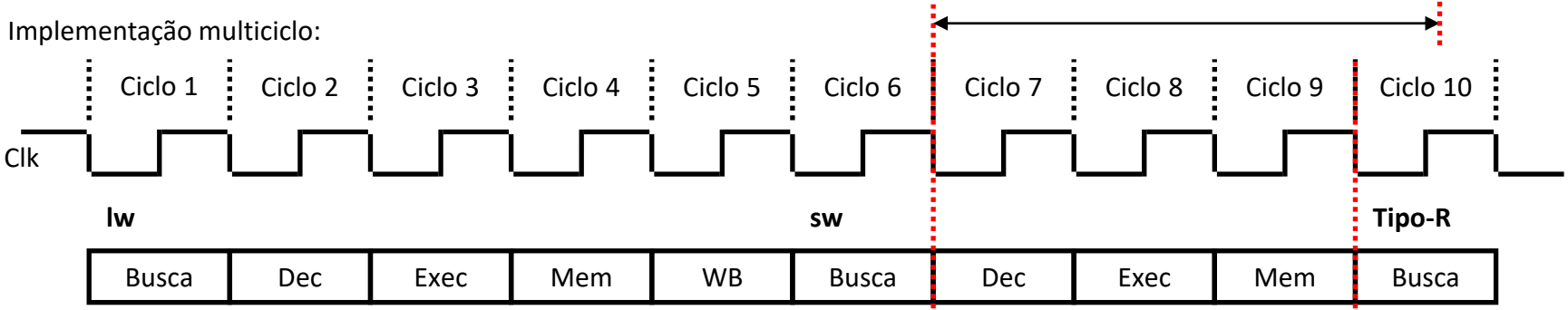


Monociclo Vs. Multiciclo

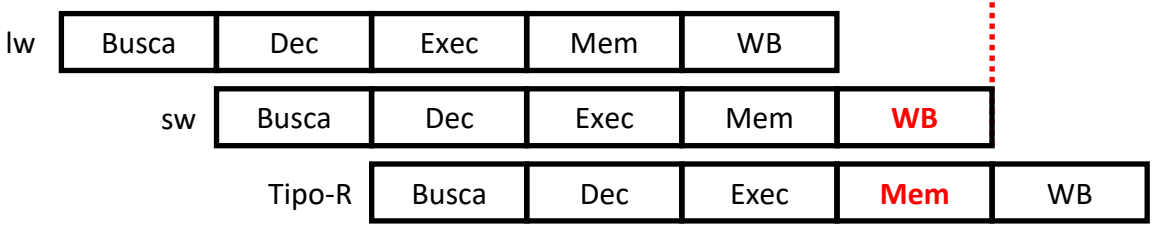
Implementação de ciclo único:



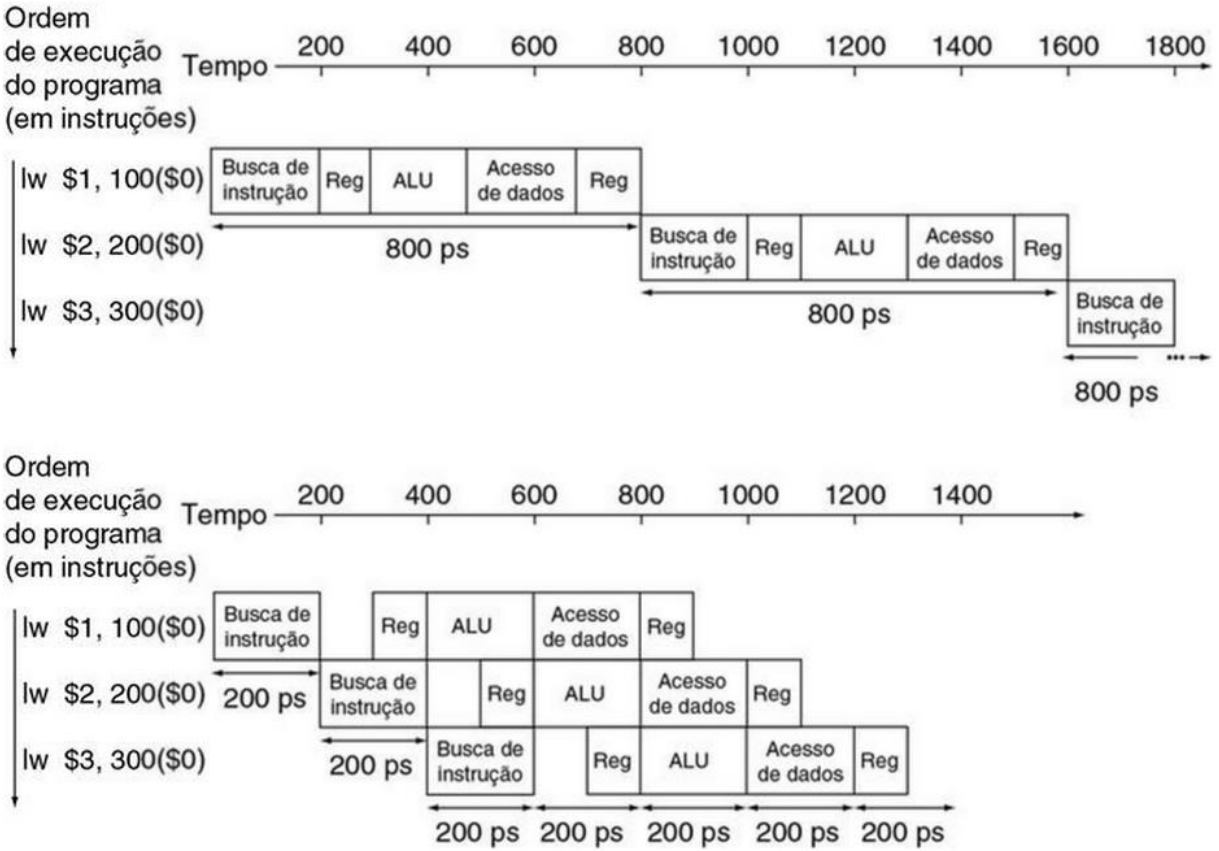
Implementação multiciclo:



Implementação Pipeline:



Ex: Pipelining



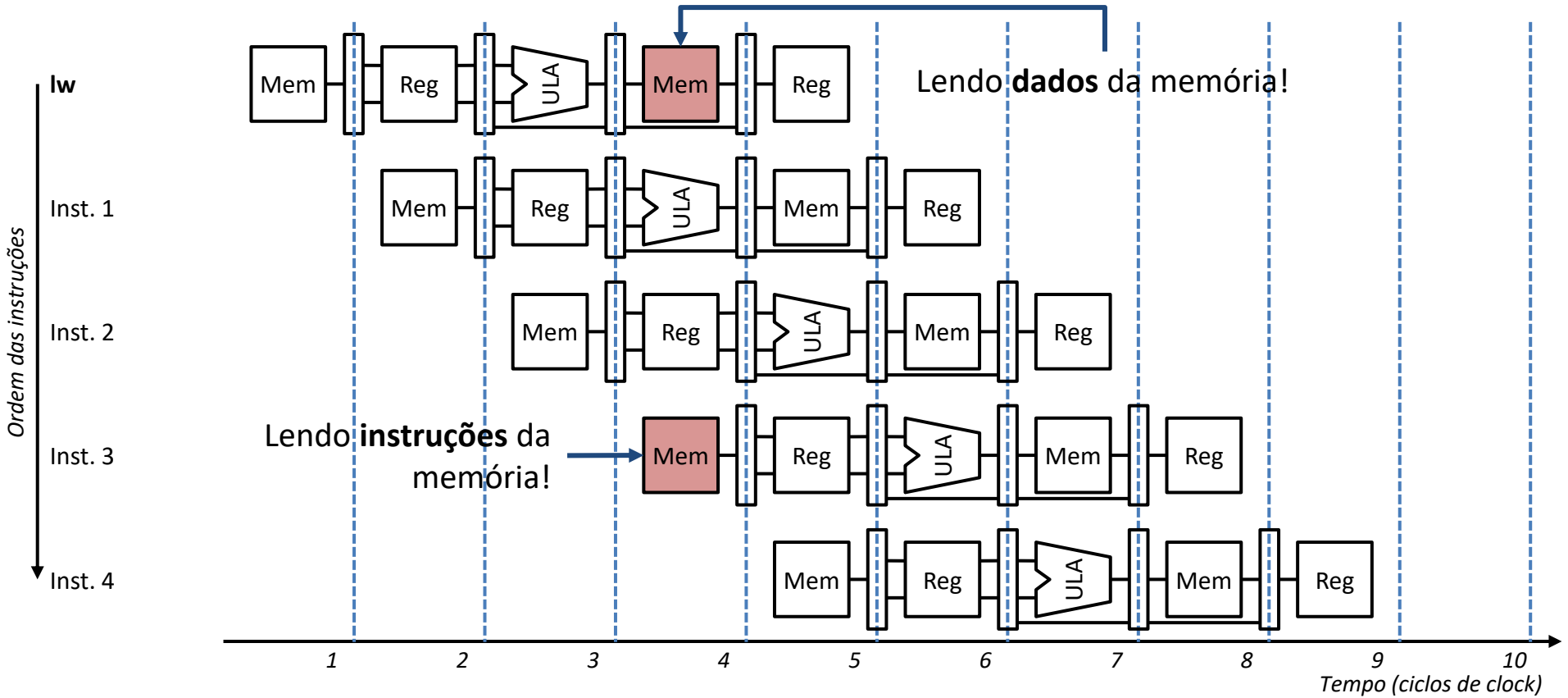
HAZARDS

Hazards

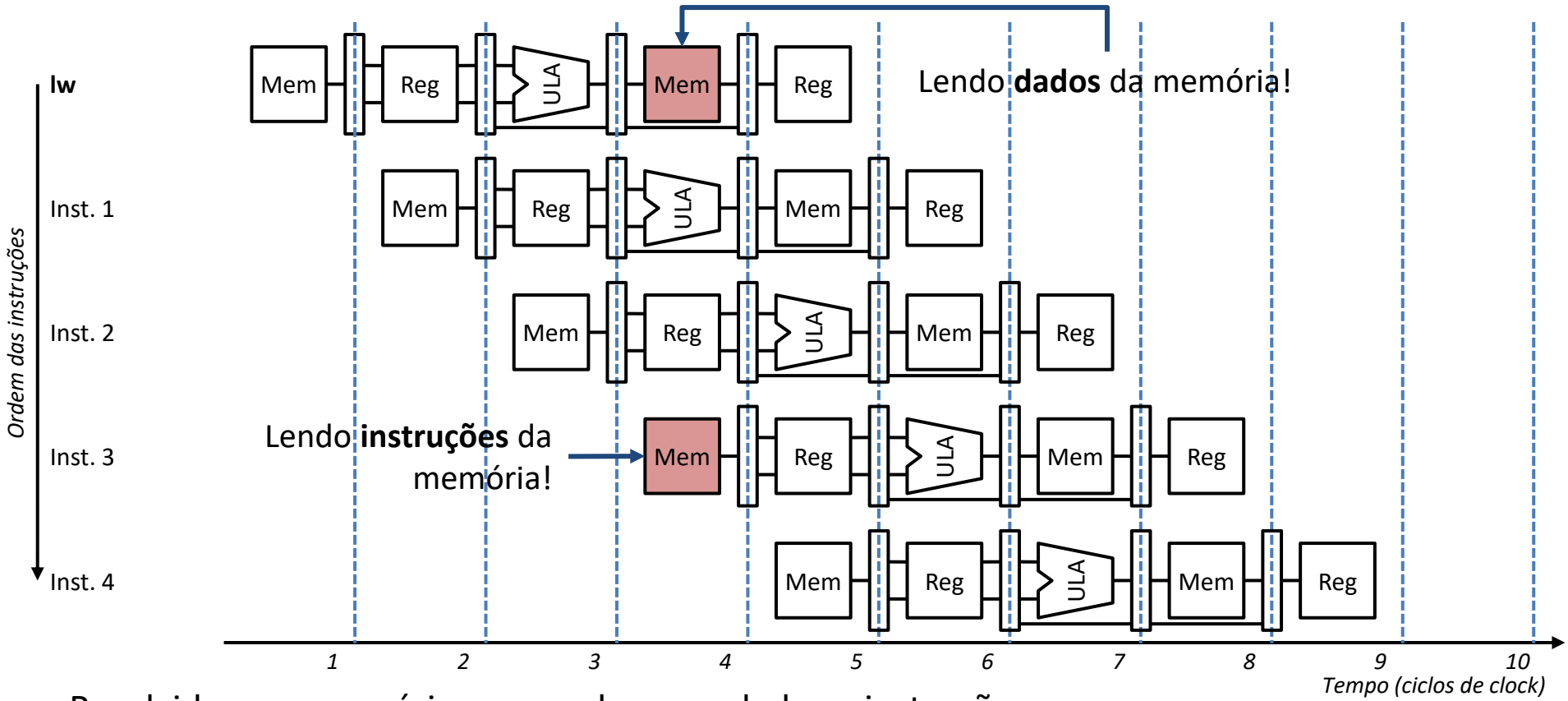
- Hazards estruturais
  - Hazard de memória simples
  - Hazard de acesso ao bando de registradores
- Hazards de dados
  - Read-before-write
  - Load-use hazard
- Hazardas de controle

HAZARDS ESTRUTURAIS

Hazard estrutural – Memória única

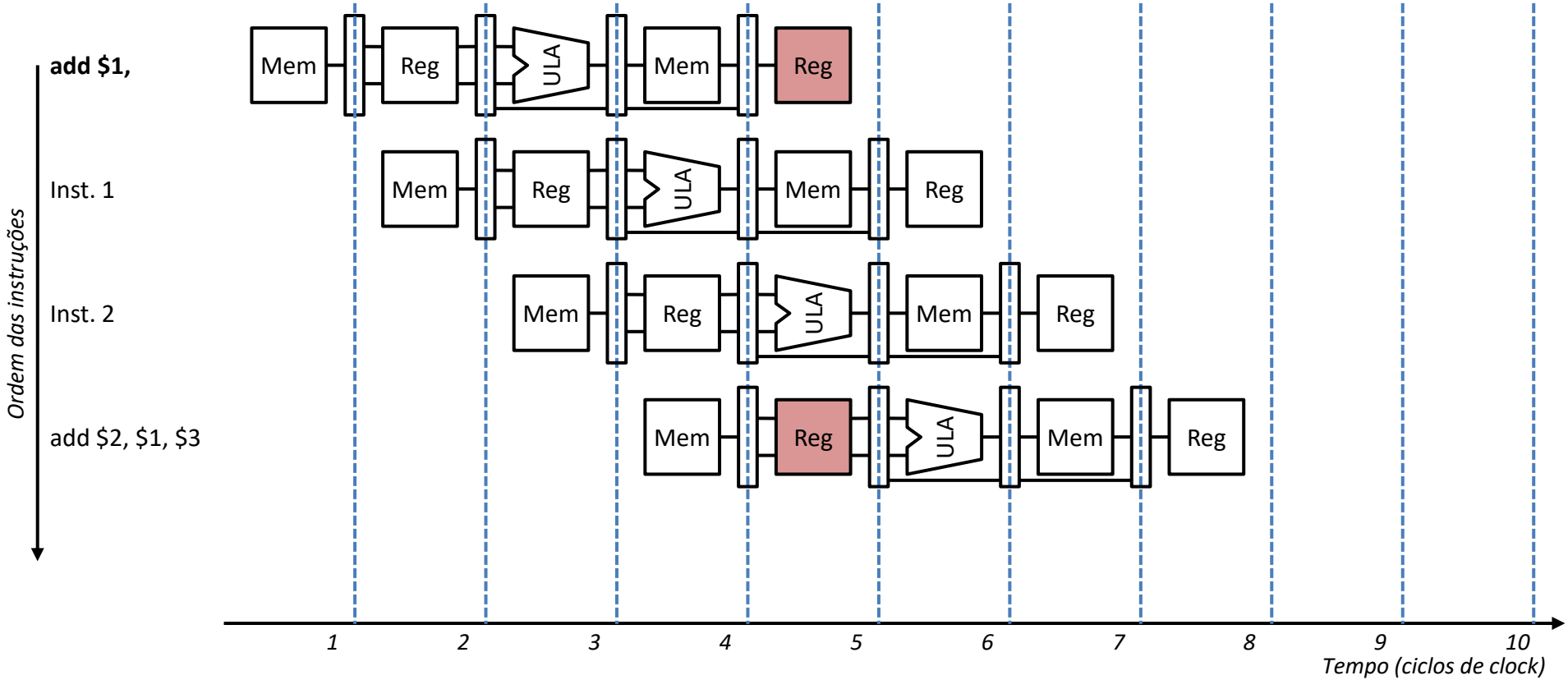


Hazard estrutural – Memória única

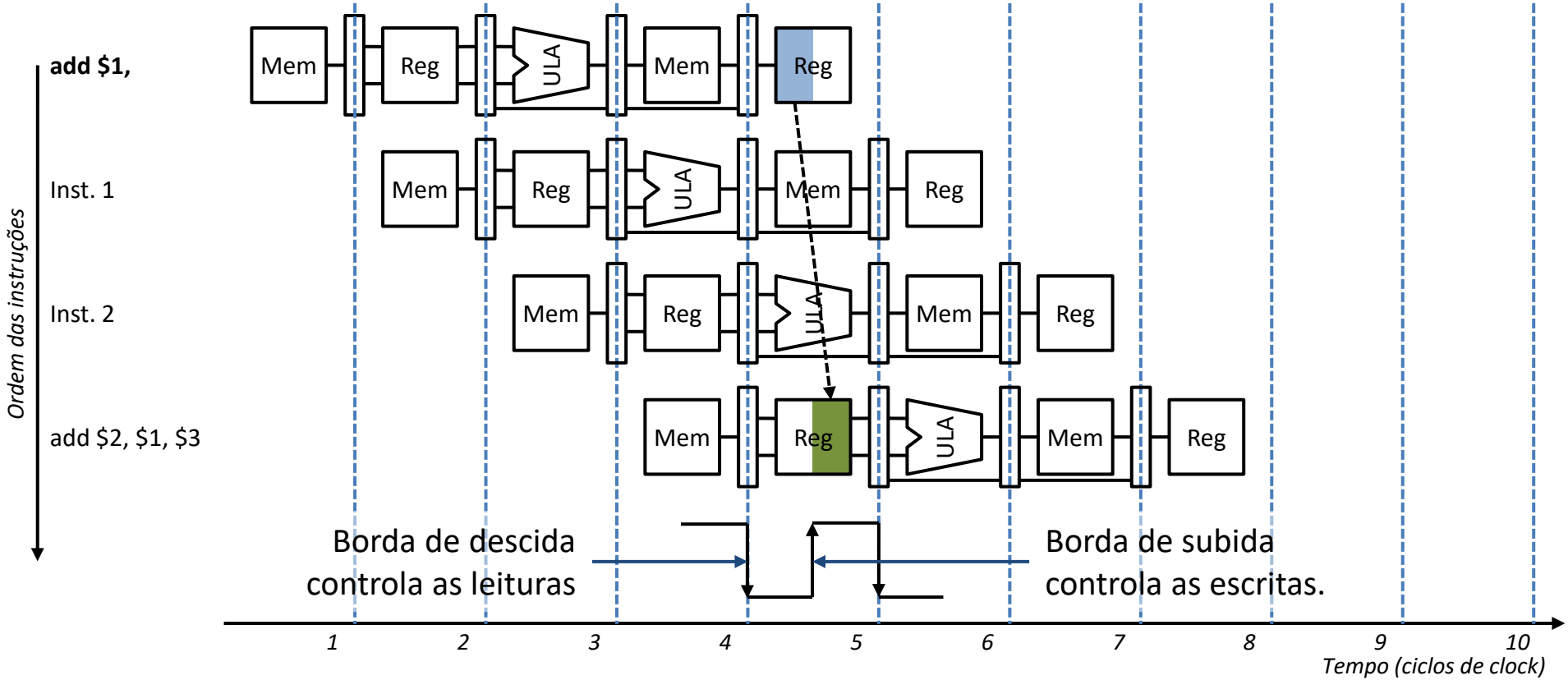


- Resolvido com memórias separadas para dados e instruções.

Hazard estrutural – Aceso ao banco de registradores



Hazard estrutural – Aceso ao banco de registradores

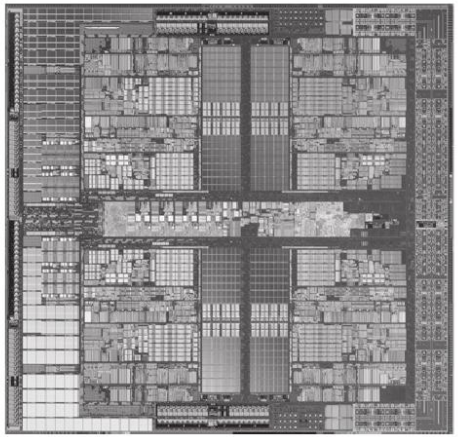


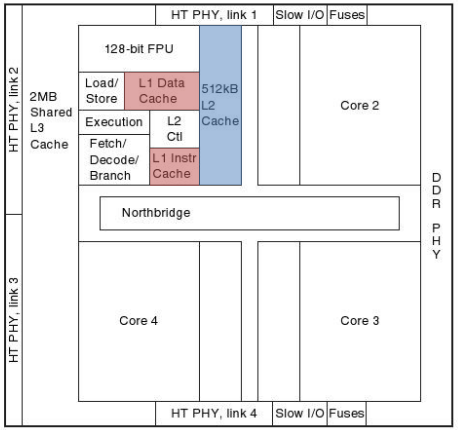
- Resolvido fazendo a leitura na segundo metade do ciclo de clock e a escrita na primeira.

Hazard estrutural – Aceso ao banco de registradores

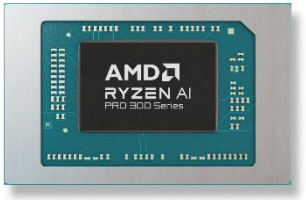
- A cache L1 (primeiro nível de cache) dos processadores é separada em cache L1 de instruções e cache L1 de dados.
  - A CPU (core) “enxerga” apenas o nível L1 da cache. A hierarquia de memórias é transparente às CPUS (cores).

AMD Barcelona





AMD Ryzen™ AI 5 PRO 340



Cache	Núcleo	Total (6 núcleos)
L1 Instr.	32 KB	192 KB
L1 Data	48 KB	288 KB
L1 Total	80 KB	480 KB
L2	1 MB	6 MB
L3 (compartilhado)	8 MB	

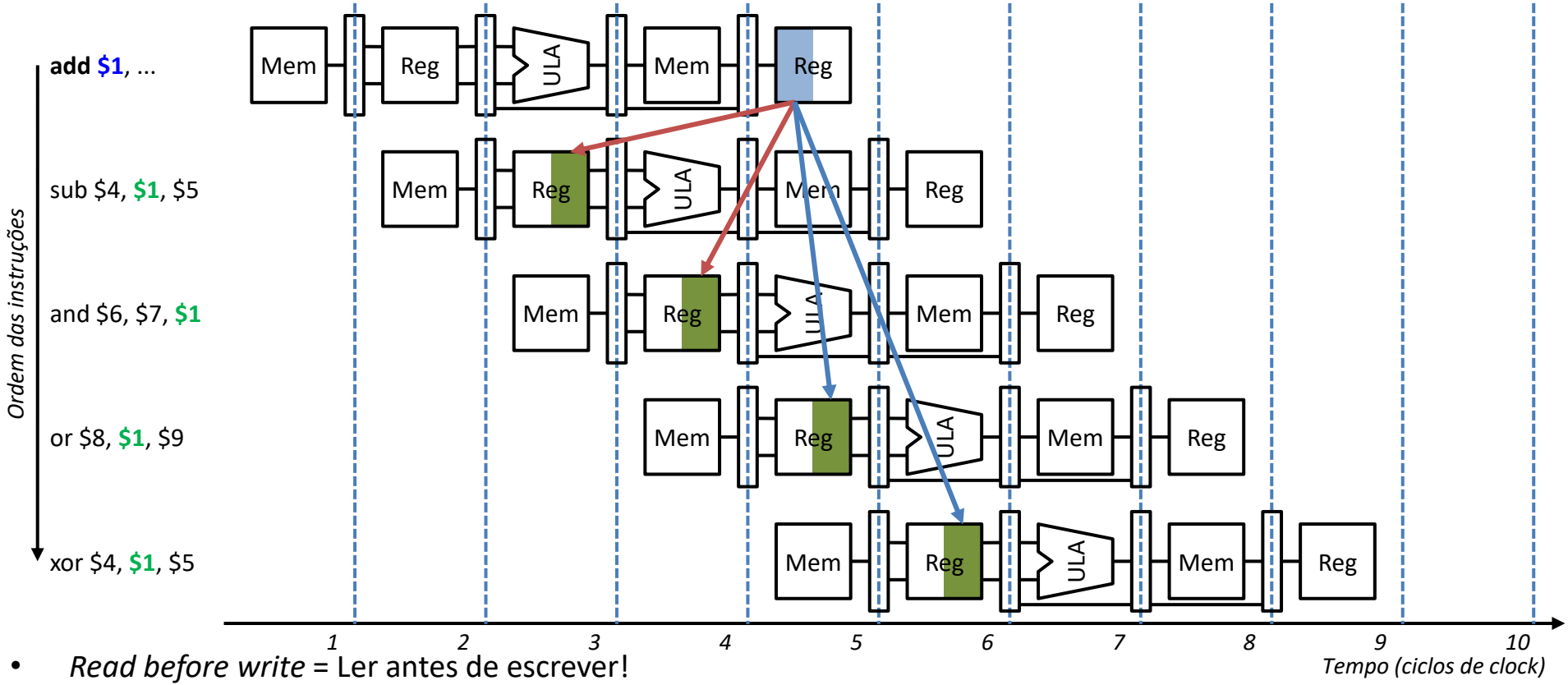
Curiosidade: Definindo o tempo de um ciclo de clock

- Um ciclo de clock deve acomodar a etapa mais lenta da execução uma instrução, que é o acesso a memória.
- Porém, a solução do hazard de acesso ao banco de registradores consiste em realizar a escrita na primeira metade e a leitura na segunda metade do ciclo de clock.
  - Dessa forma, um ciclo de clock também deve acomodar dois acessos ao banco de registradores.
- O acesso ao banco de registradores é muito mais rápido do que o acesso à memória, aproximadamente 2 vezes mais rápido.
  - Então, a duração de um ciclo de clock deve ser o que durar mais tempo.

```
Se tempo(MEM) >= 2 x tempo(REG):
     tempo(CC) = tempo(MEM)
Senão:   // tempo(MEM) < 2 x tempo(REG):
     tempo(CC) = 2 x tempo(REG)
```

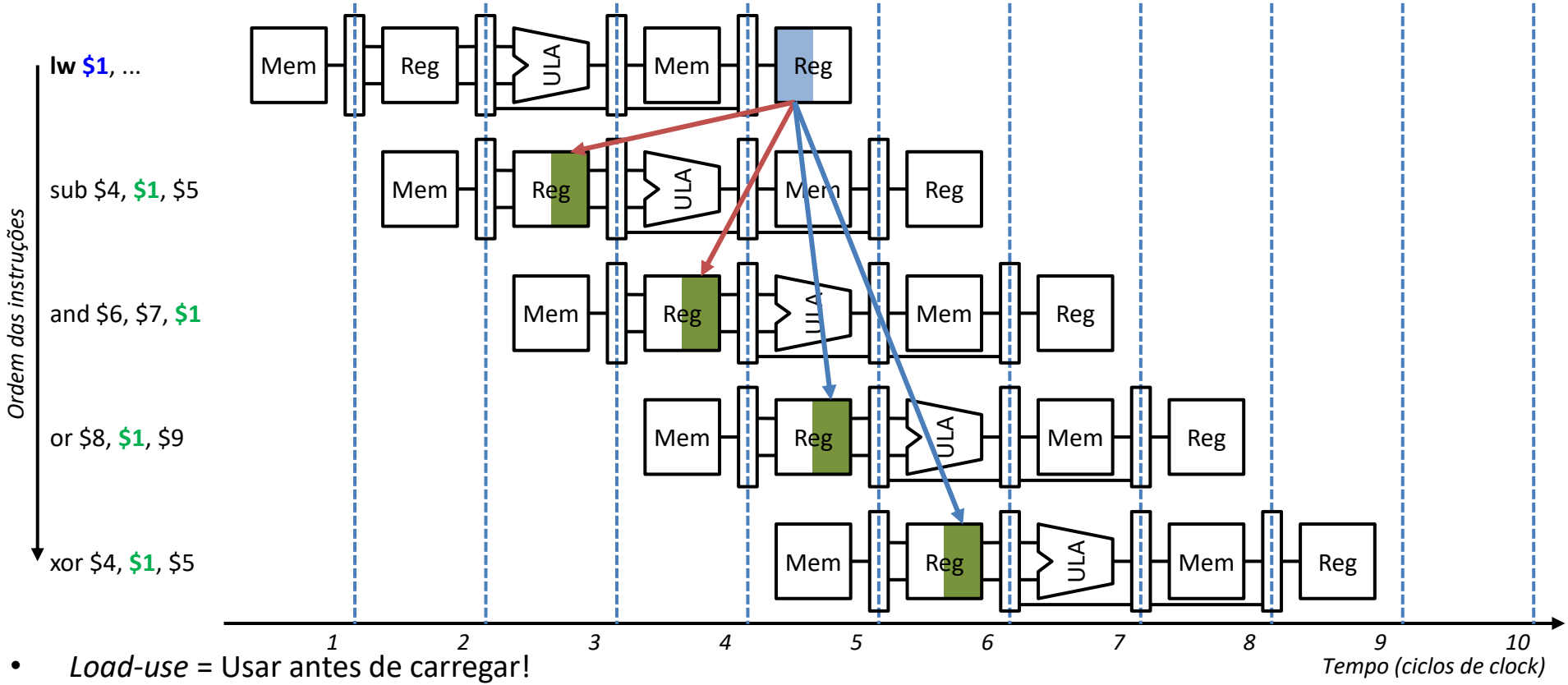
HAZARDS DE DADOS

Hazard de dados – *Read before write*



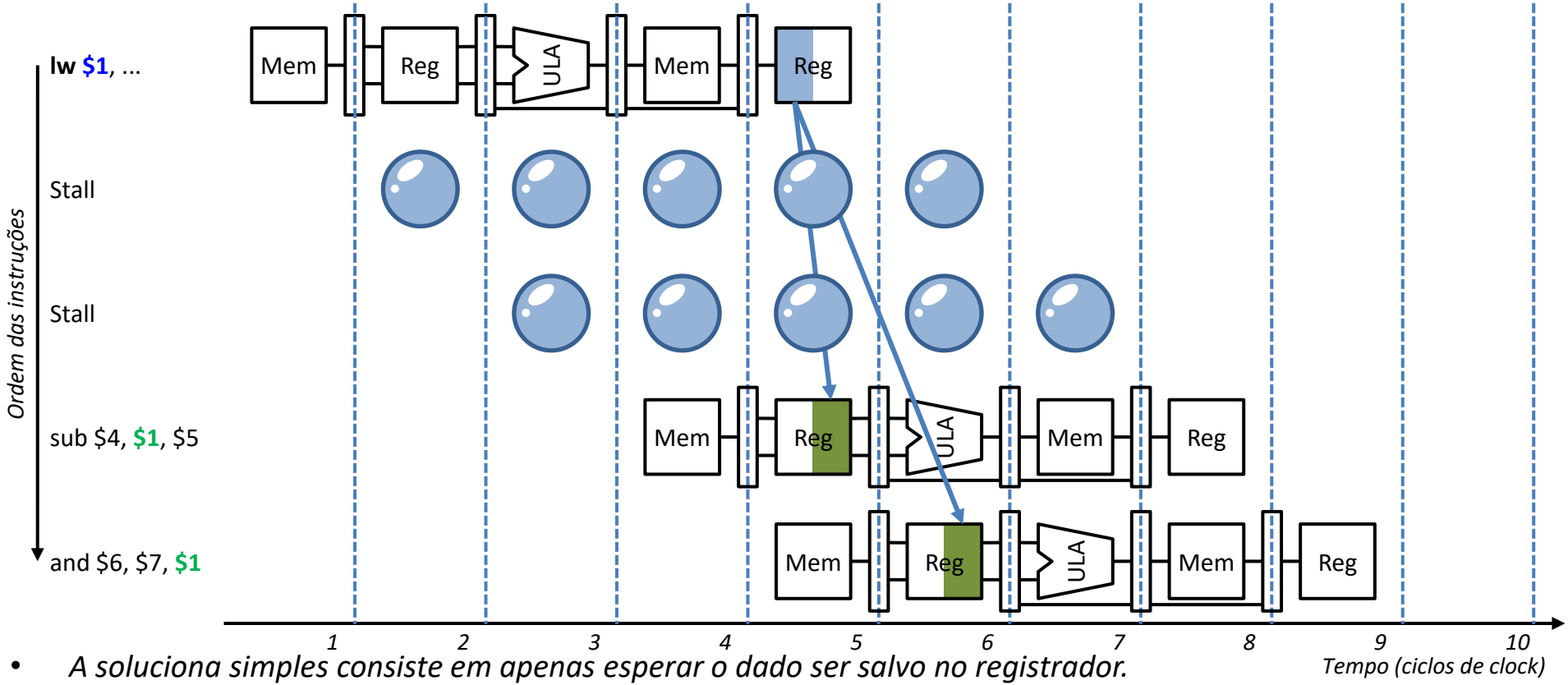
- *Read before write* = Ler antes de escrever!
  - Pode se gerado (iniciado) por instruções lógicas/aritméticas!

Hazard de dados – *Load-use hazard*



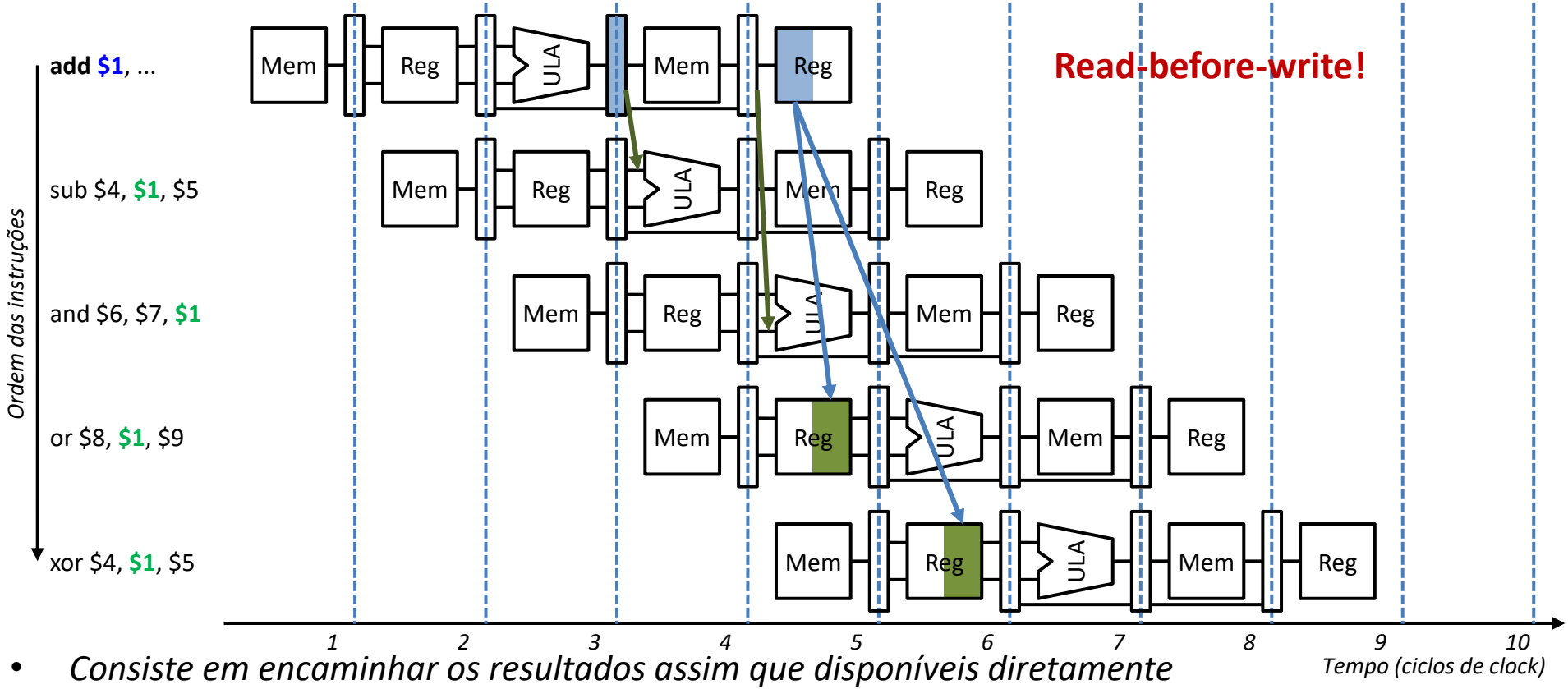
- *Load-use* = Usar antes de carregar!
  - Pode se gerado (iniciado) por load word (lw)!

Hazard de dados – Solução simples



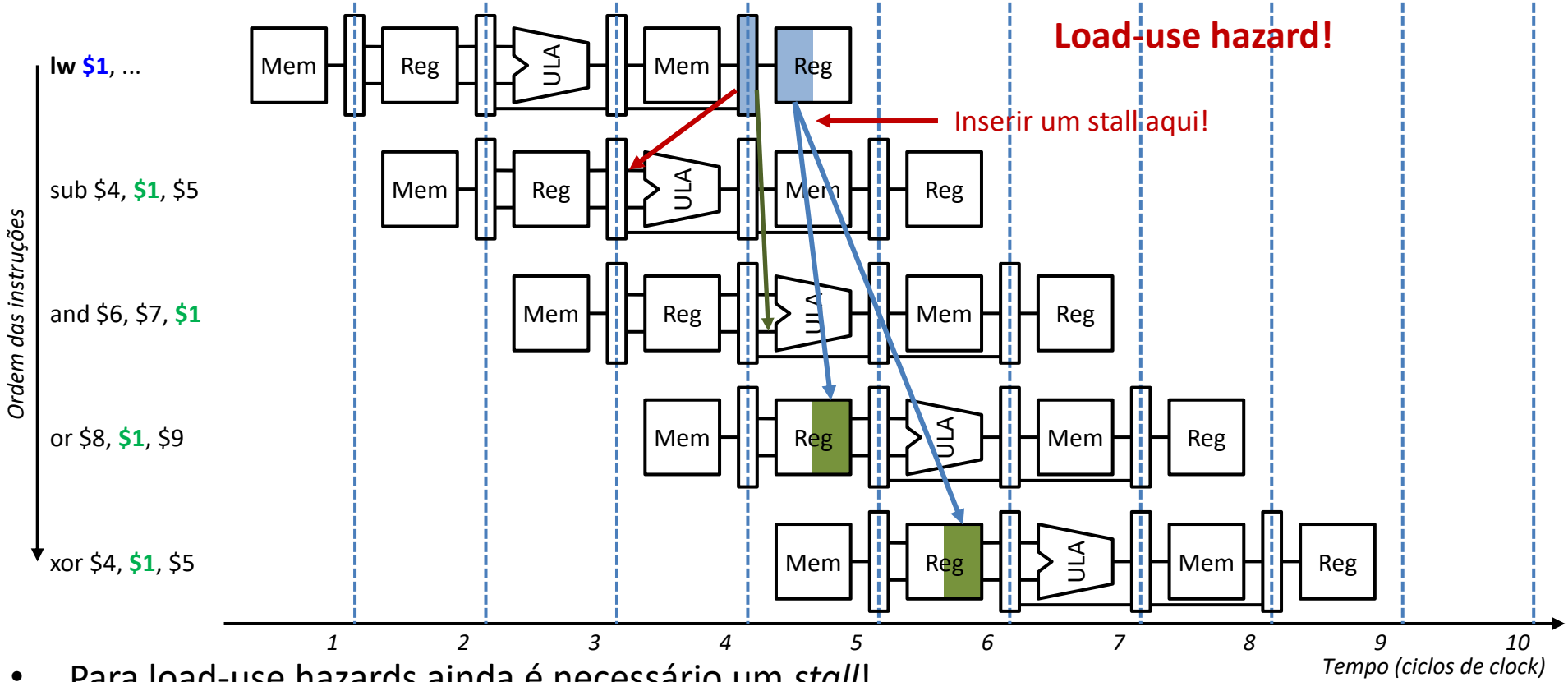
- *A soluciona simples consiste em apenas esperar o dado ser salvo no registrador.*
  - *Esta solução impacta o CPI, reduzindo o desempenho.*

Hazard de dados – Solução por encaminhamento



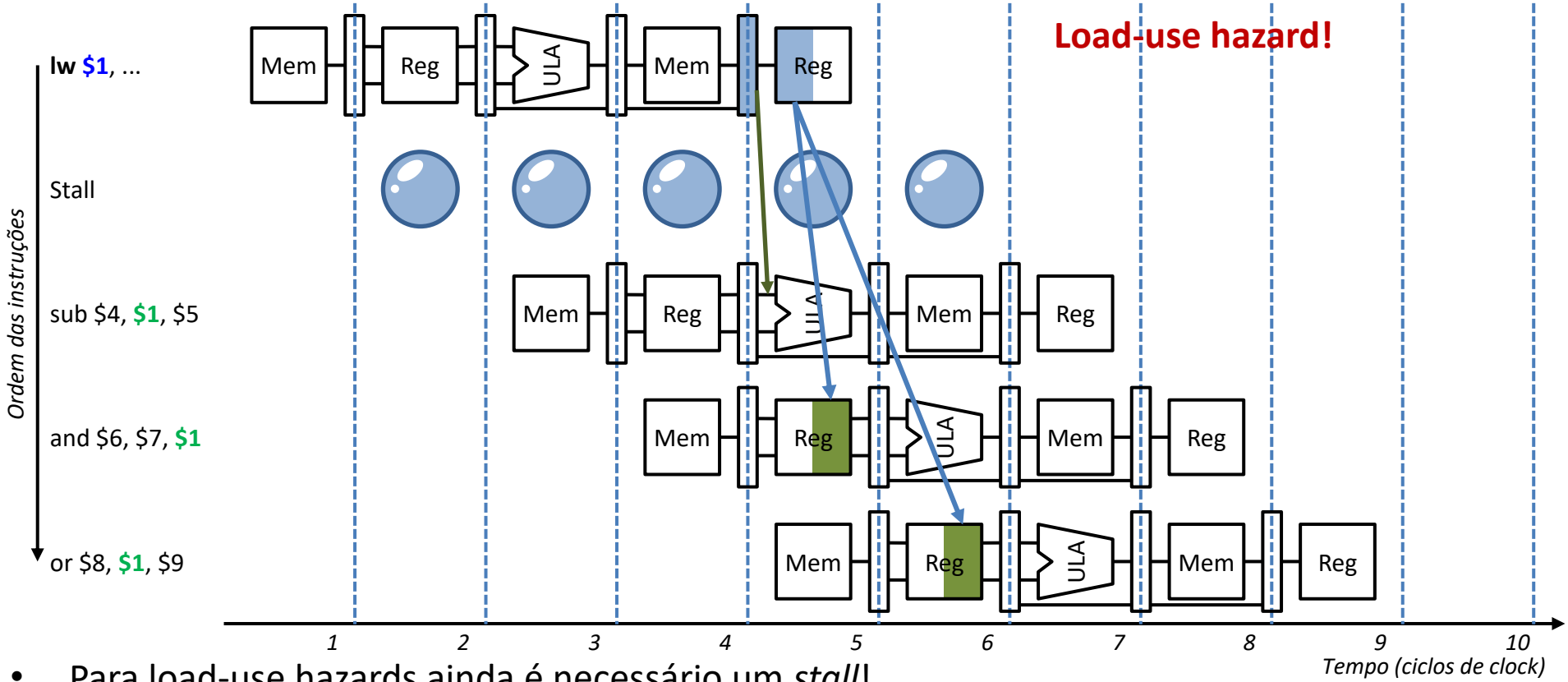
- *Consiste em encaminhar os resultados assim que disponíveis diretamente para onde forem necessários*

Hazard de dados – Solução por encaminhamento



- Para load-use hazards ainda é necessário um *stall*!

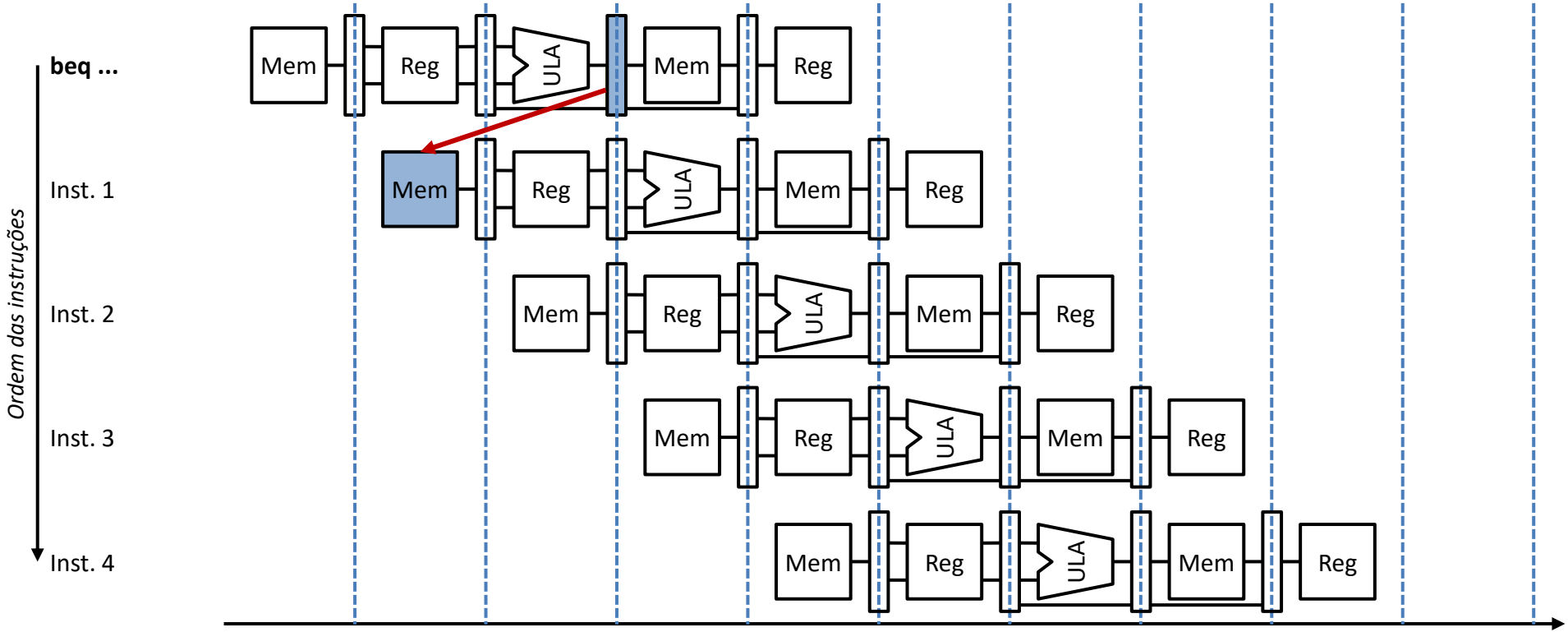
Hazard de dados – Solução por encaminhamento



- Para load-use hazards ainda é necessário um *stall*!

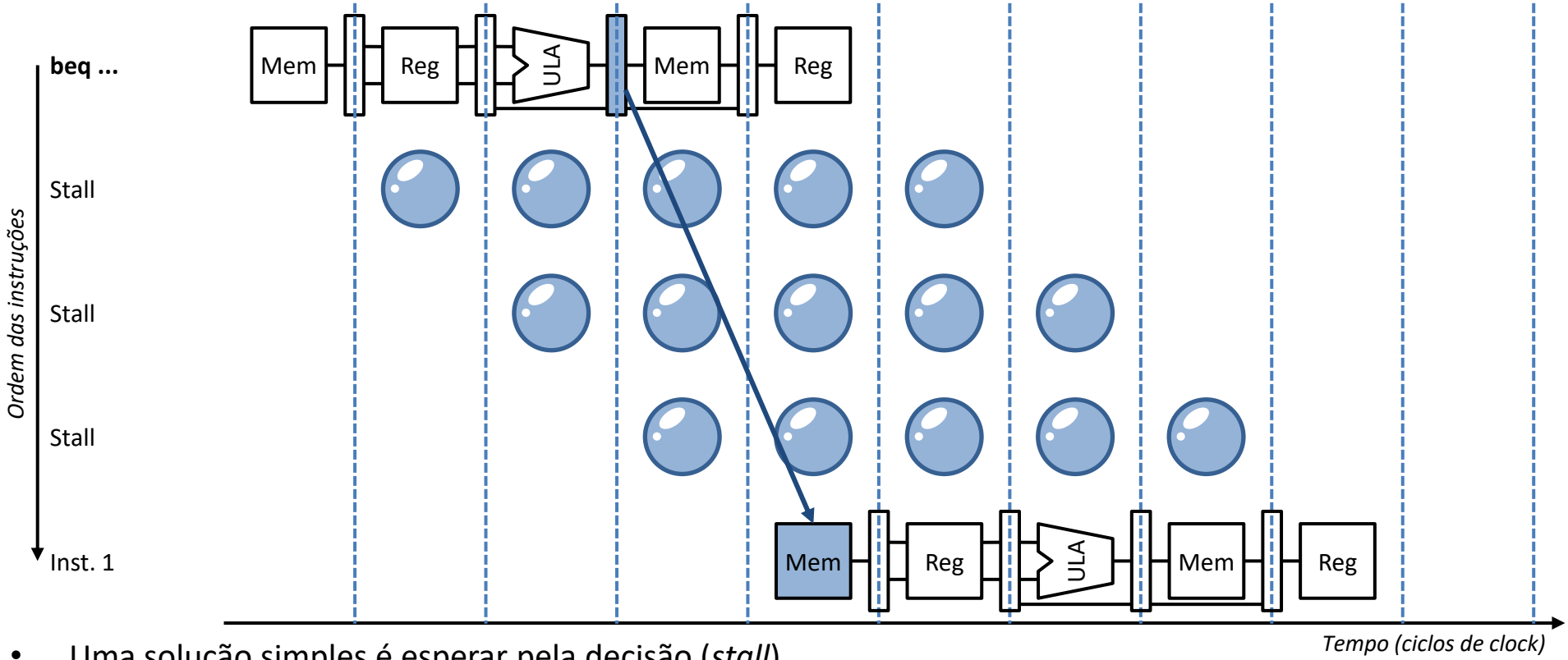
HAZARDS DE CONTROLE

Hazard de controle



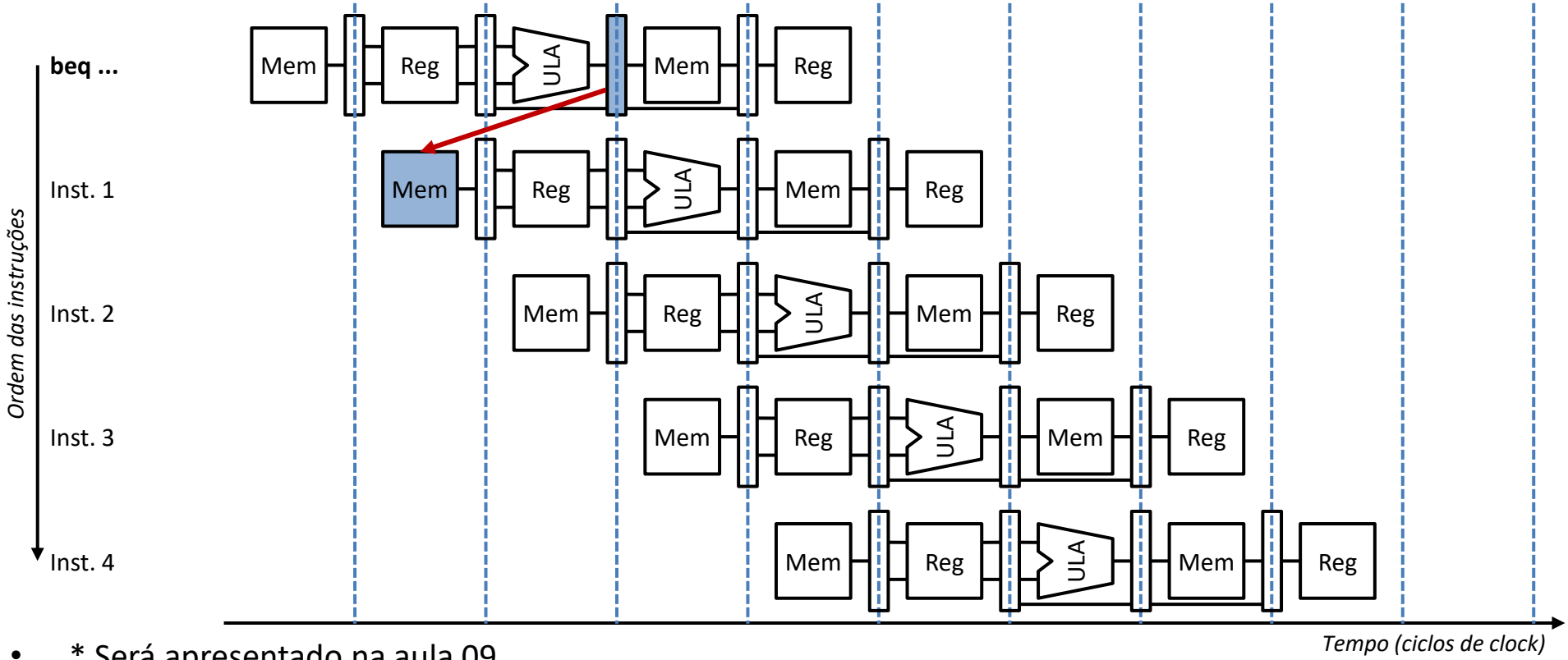
- O resultado do teste condicional está disponível apenas após a etapa de execução.
  - Como saber qual instrução carregar? No caso de um *if*, do bloco **True** ou do bloco **False**?

Hazard de controle – Solução simples



- Uma solução simples é esperar pela decisão (*stall*).
  - Esta solução impacta o CPI. É uma péssimo solução.

Hazard de controle – Previsão de desvios



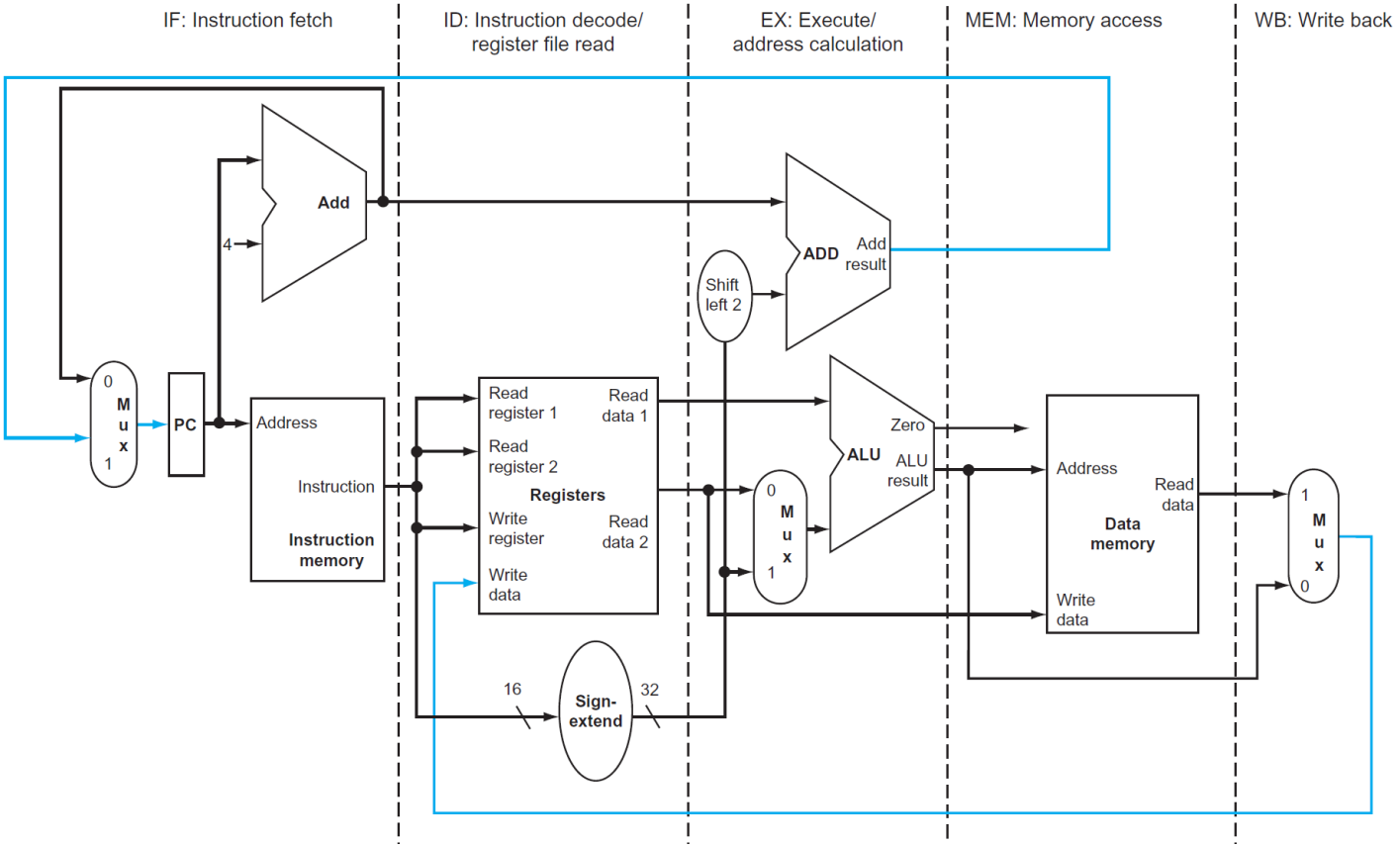
- * Será apresentado na aula 09.

MIPS PIPELINE – CAMINHO DE DADOS E CONTROLE

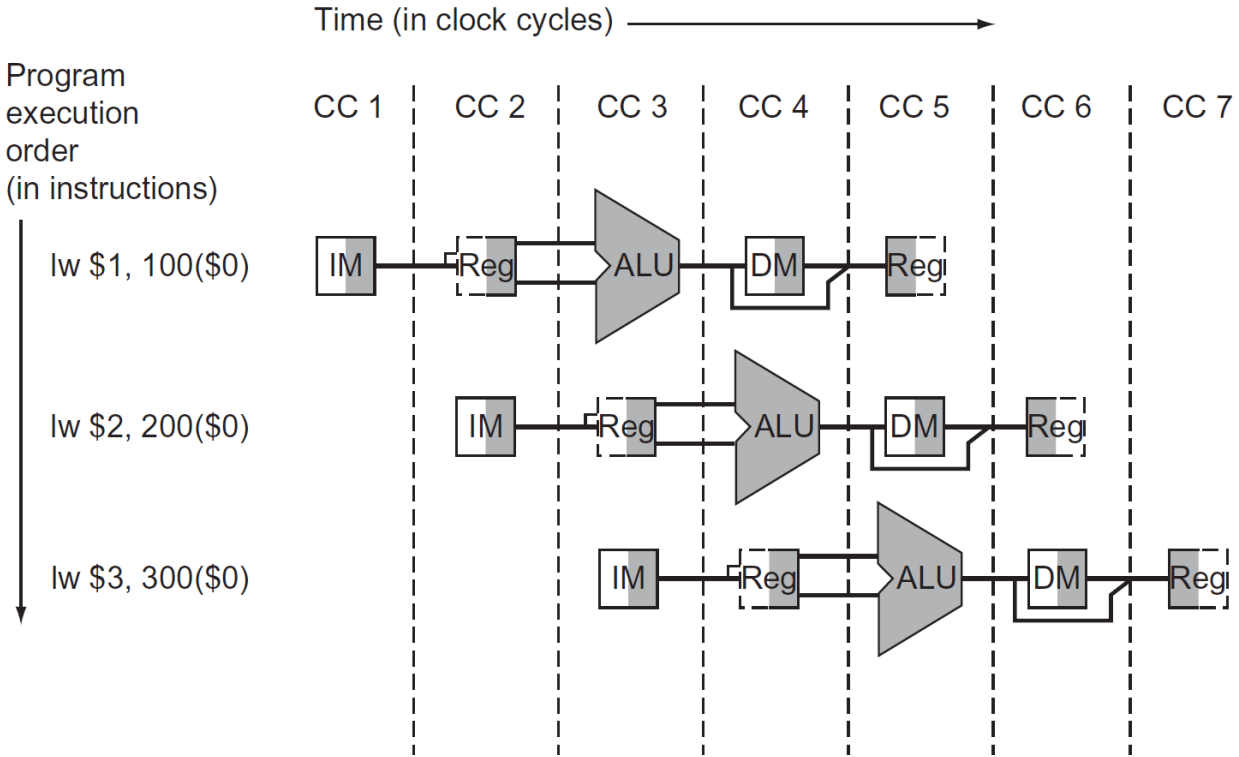
Divisão de instrução em cinco estágios

- Divisão de uma instrução em cinco estágios:
  - Cinco níveis de pipeline
  - Cinco instruções executadas em paralelo (durante o mesmo ciclo de clock)
    - **IF (Instruction Fetch):** Busca de instruções
    - **ID (Instruction Decode):** Decodificação de instruções e leitura do banco de registradores
    - **EX (Execution):** Execução ou calculo de endereço
    - **MEM (Memory):** Acesso à memória de dados
    - **WB (Write Back):** Escrita do resultado

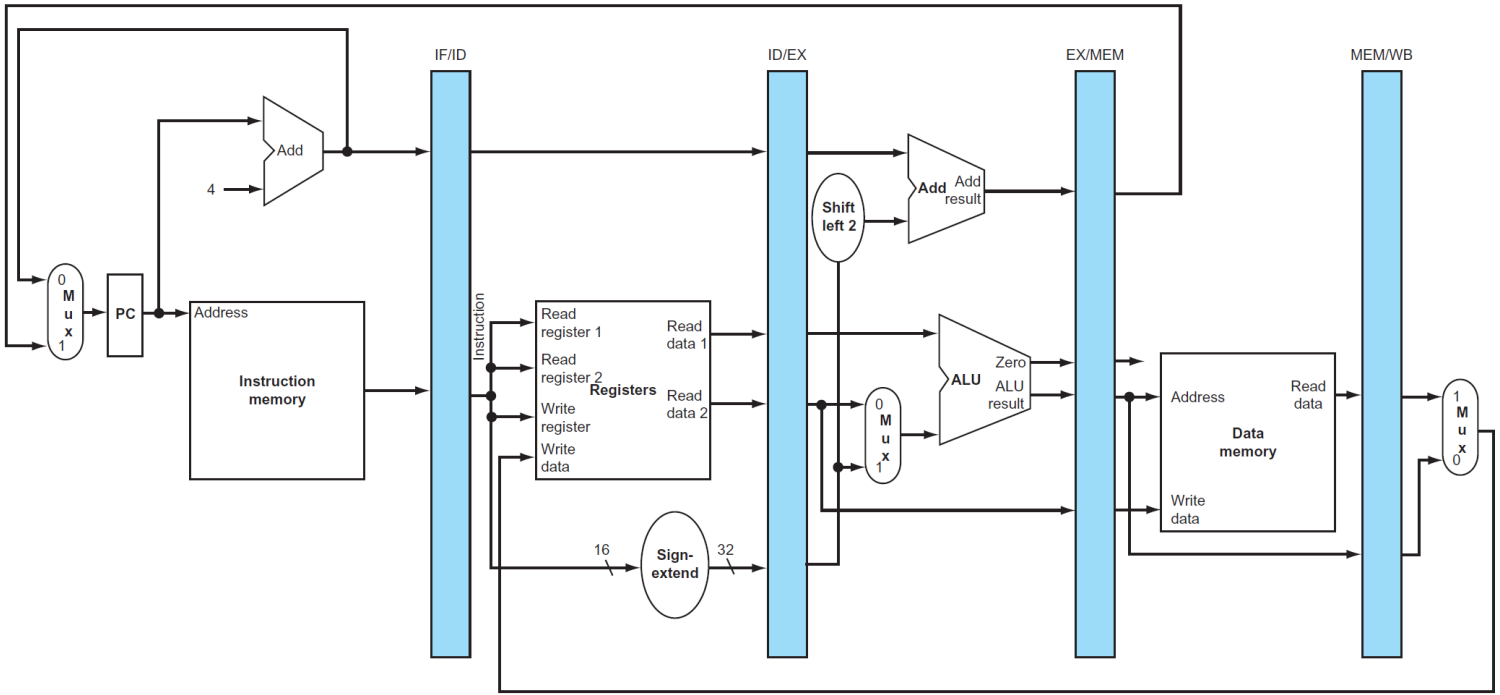
Divisão de instrução em cinco estágios



Divisão de instrução em cinco estágios



Divisão de instrução em cinco estágios

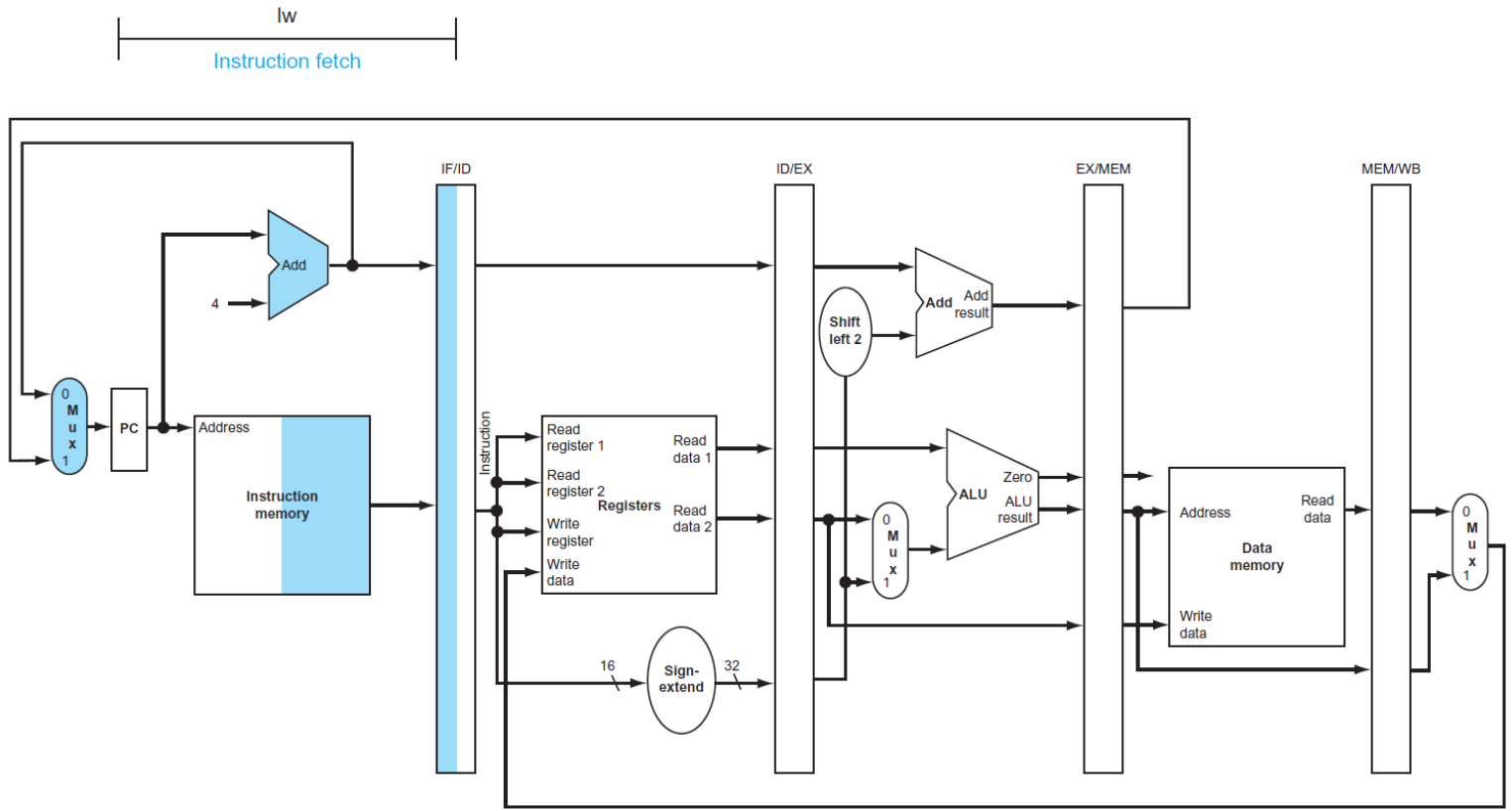


- IF/ID (64 bits) – instrução (32 bits) + endereço do PC++ (32 bits)
- ID/EX (128 bits) – PC++ (32 bits) + Dado da leitura 1 (32 bits) + Dado da leitura 2 (32 bits) + Offset estendido (32 bits)
- EX/MEM (97 bits) – PC++ + desvio (32 bits) + Flag zero (1 bit) + Resultado da ALU (32 bits) + Dado da leitura 2 (32 bits)
- MEM/WB (64 bits) – Dados lidos da memória (32 bits) + Resultado da ALU (32 bits)
  - *Os registradores ainda serão expandidos no futuro!!!*

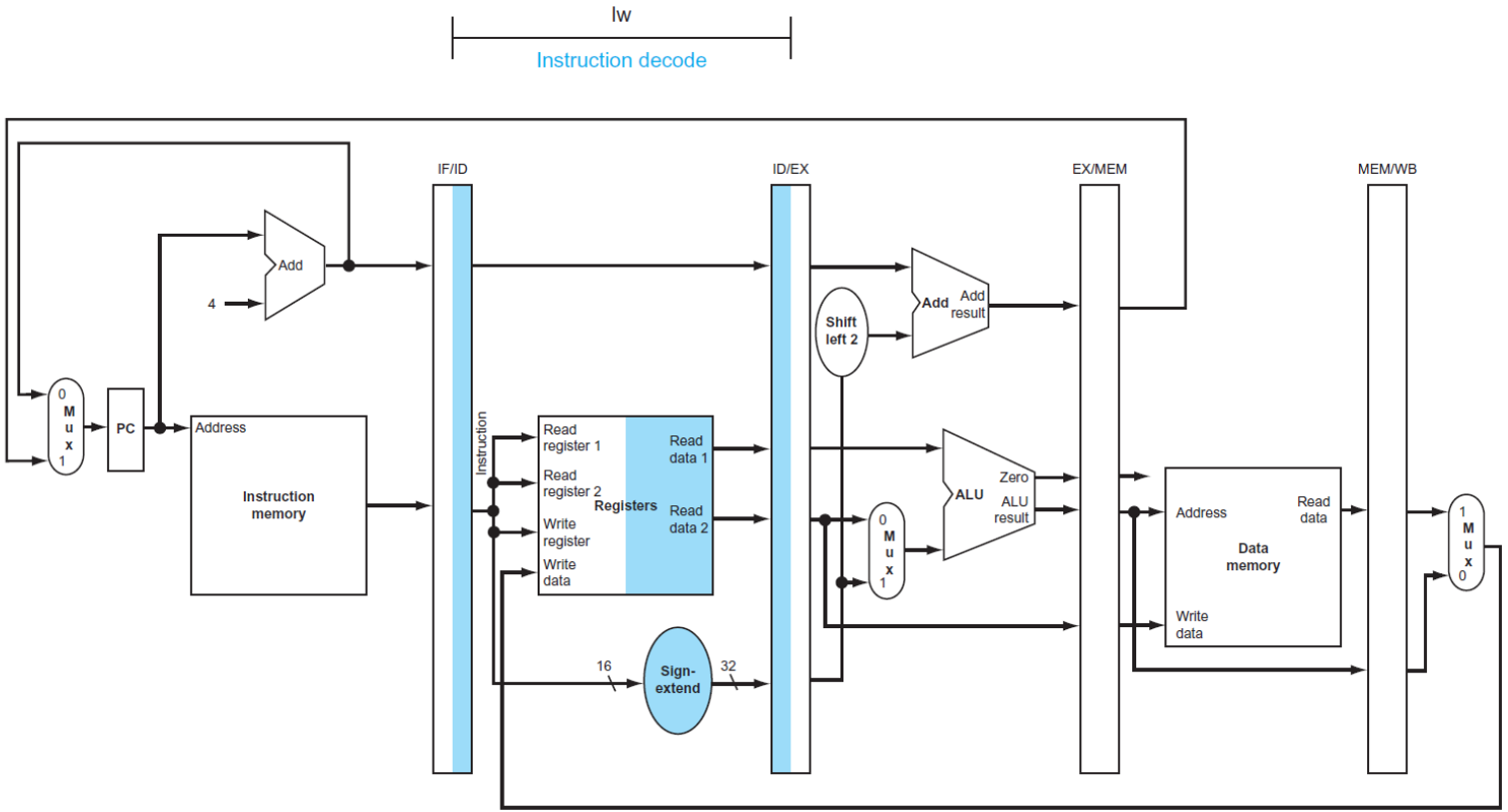
Instrução load word (lw)

- Etapa 1:
  - **IF** - Busca da instrução
- Etapa 2:
  - **ID -** Decodificação de instruções e leitura do banco de registradores
- Etapa 3:
  - **EX -** Execução ou calculo de endereço
- Etapa 4:
  - **MEM -** Acesso a memória
- Etapa 5:
  - **WB** - Escrita de resultado

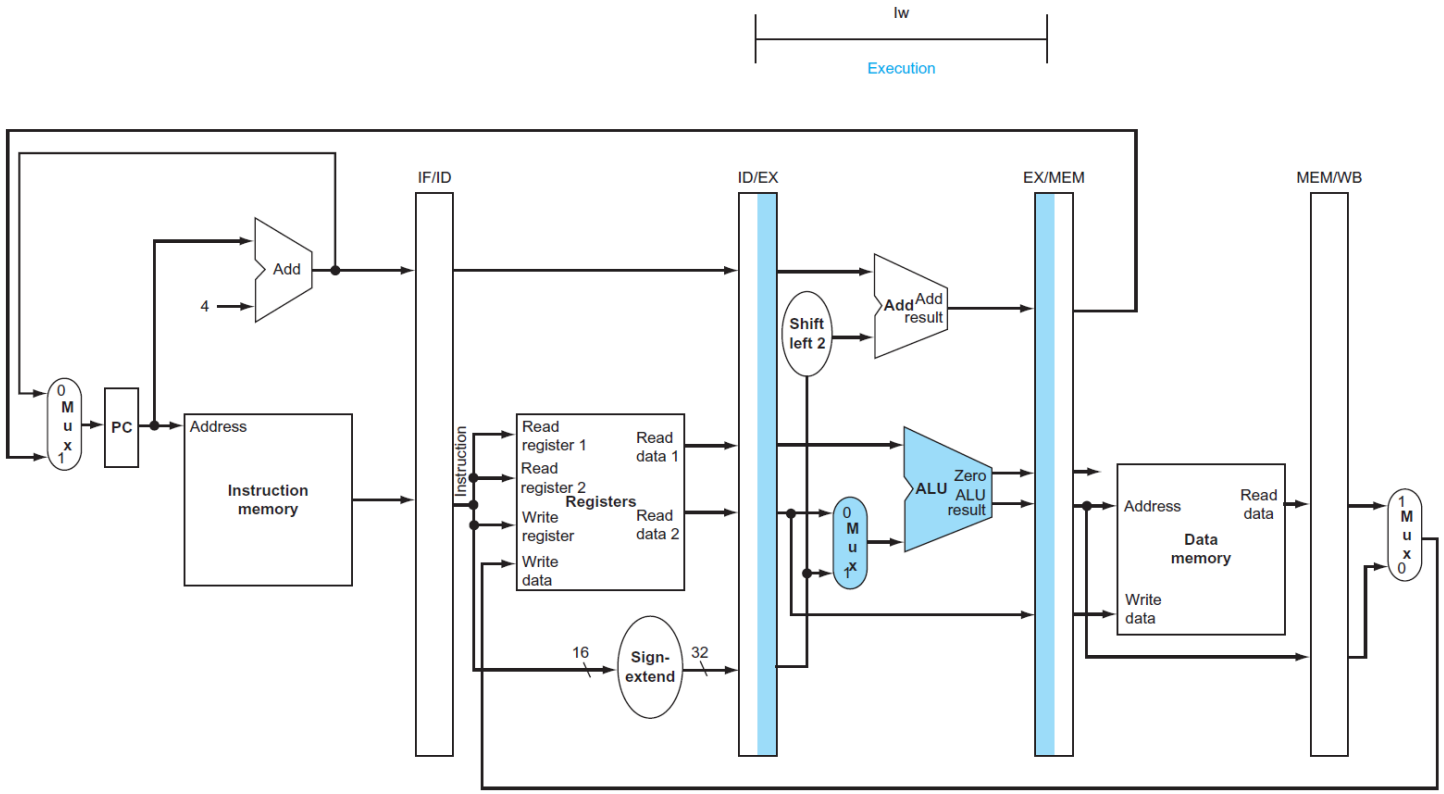
Etapa 1: IF – Busca da instrução



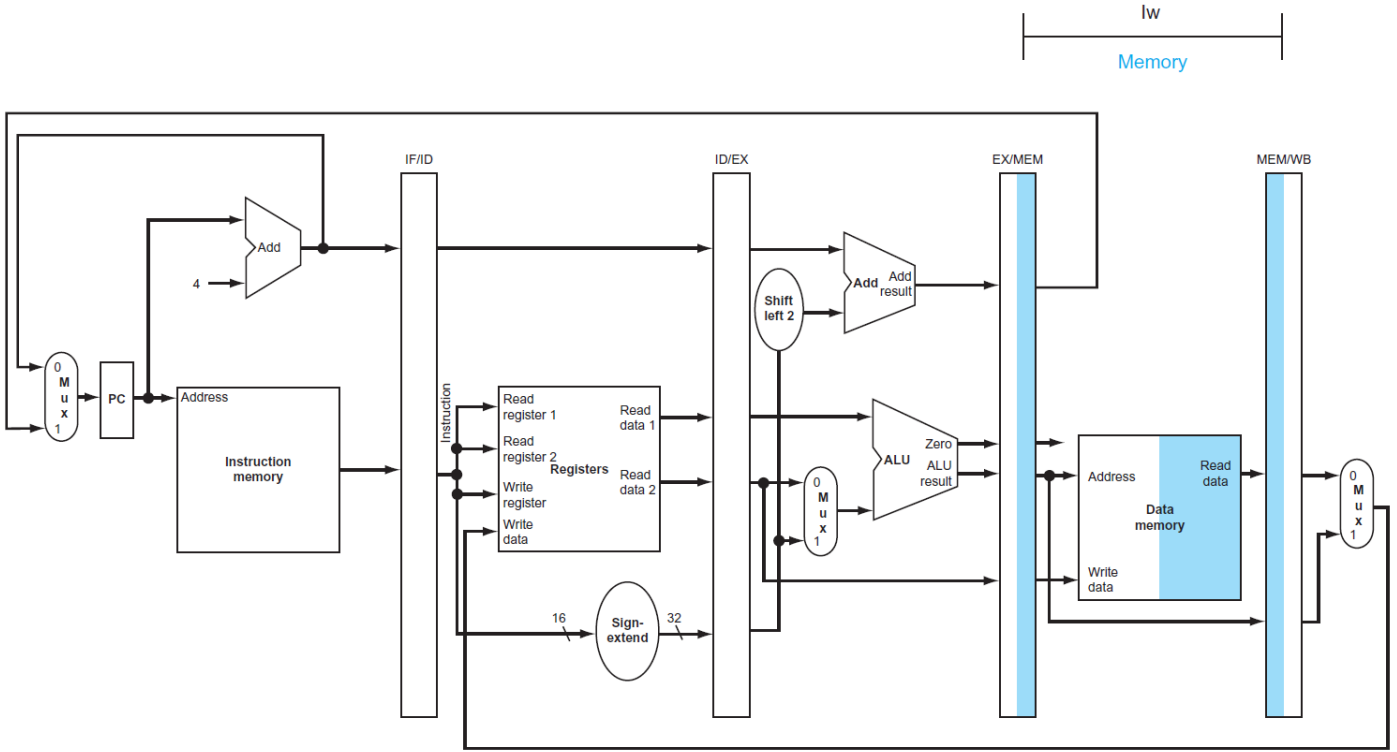
Etapa 2: ID – Decodificação da instrução



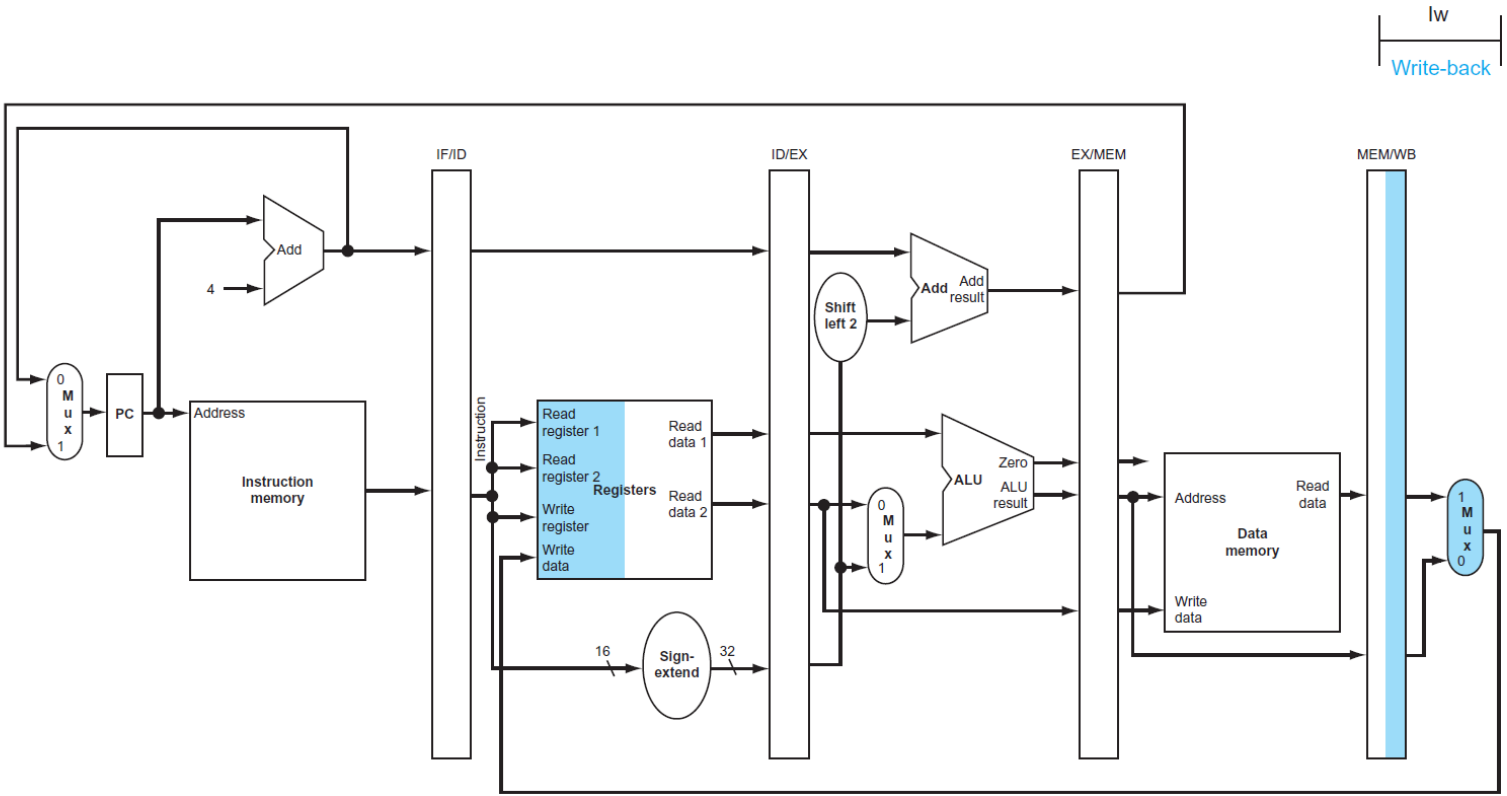
Etapa 3: EX – Execução da instrução (lw)



Etapa 4: MEM – Acesso à memória (lw)



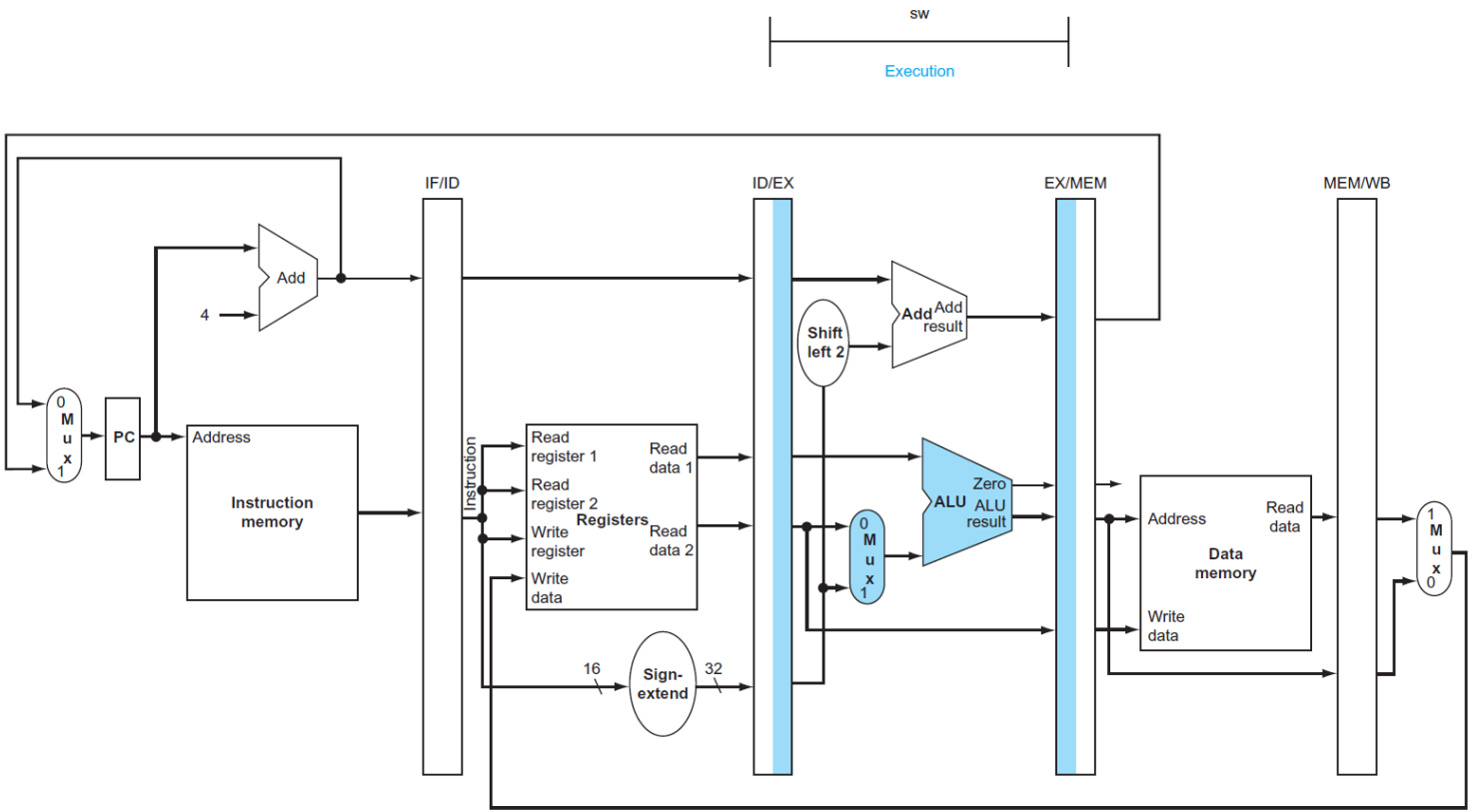
Etapa 5: WB – Escreve nos registradores (lw)



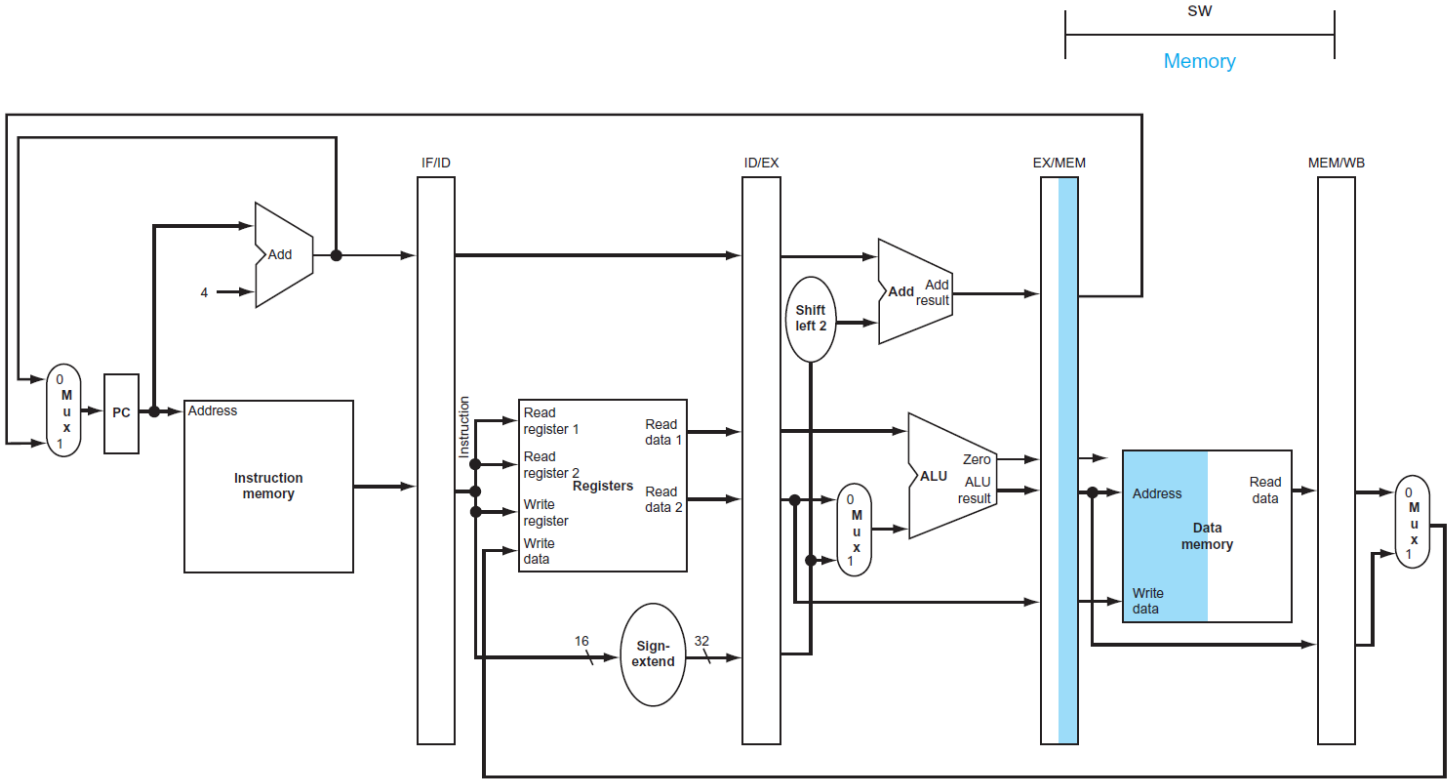
Instrução store word (sw)

- Etapa 1:
  - **IF -** Busca de instruções
- Etapa 2:
  - **ID -** Decodificação de instruções e leitura do banco de registradores
- Etapa 3:
  - **EX -** Execução ou calculo de endereço
- Etapa 4:
  - **MEM -** Acesso a memória
- Etapa 5:
  - **WB -** Escrita de resultado

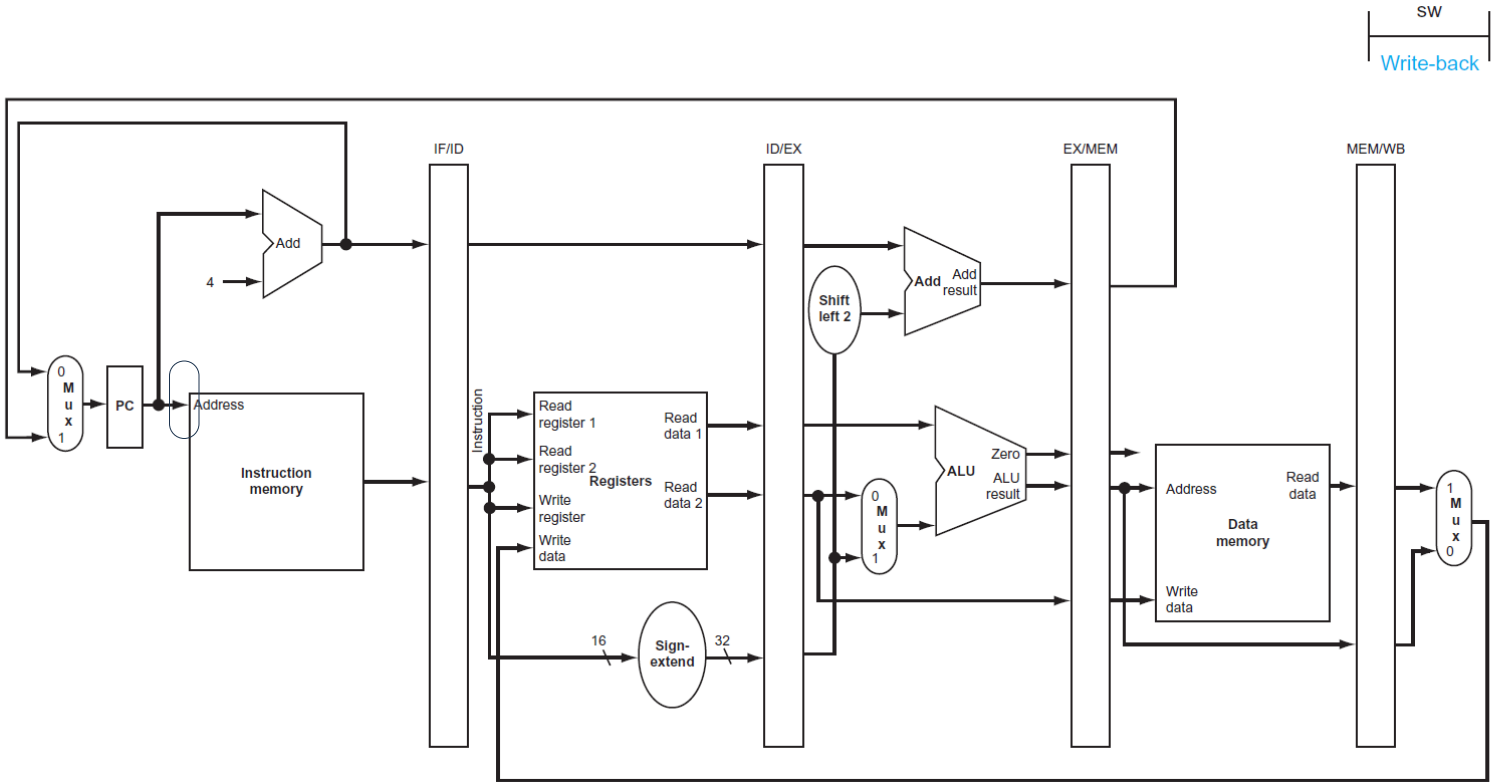
Etapa 3: EX – Execução (sw)



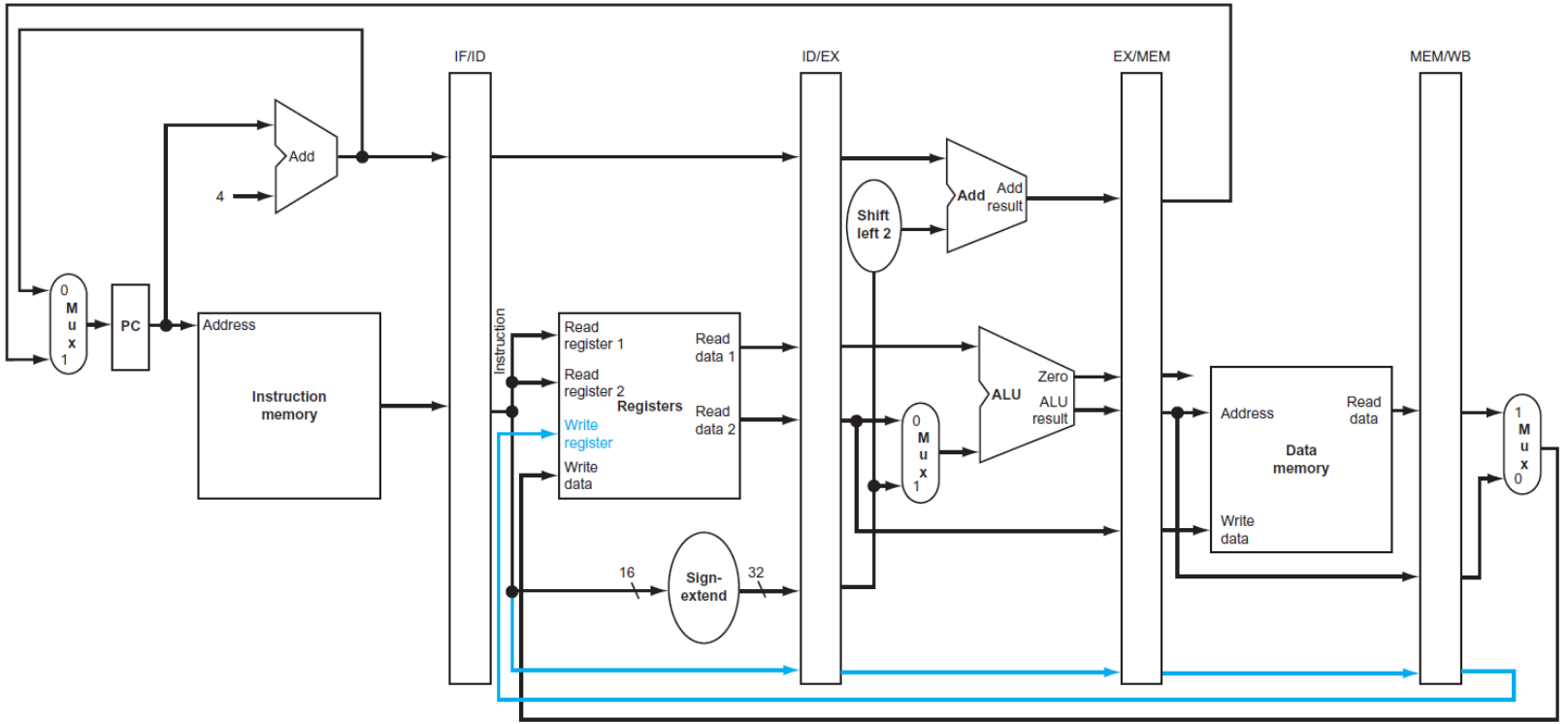
Etapa 4: MEM – Acesso à memória (sw)



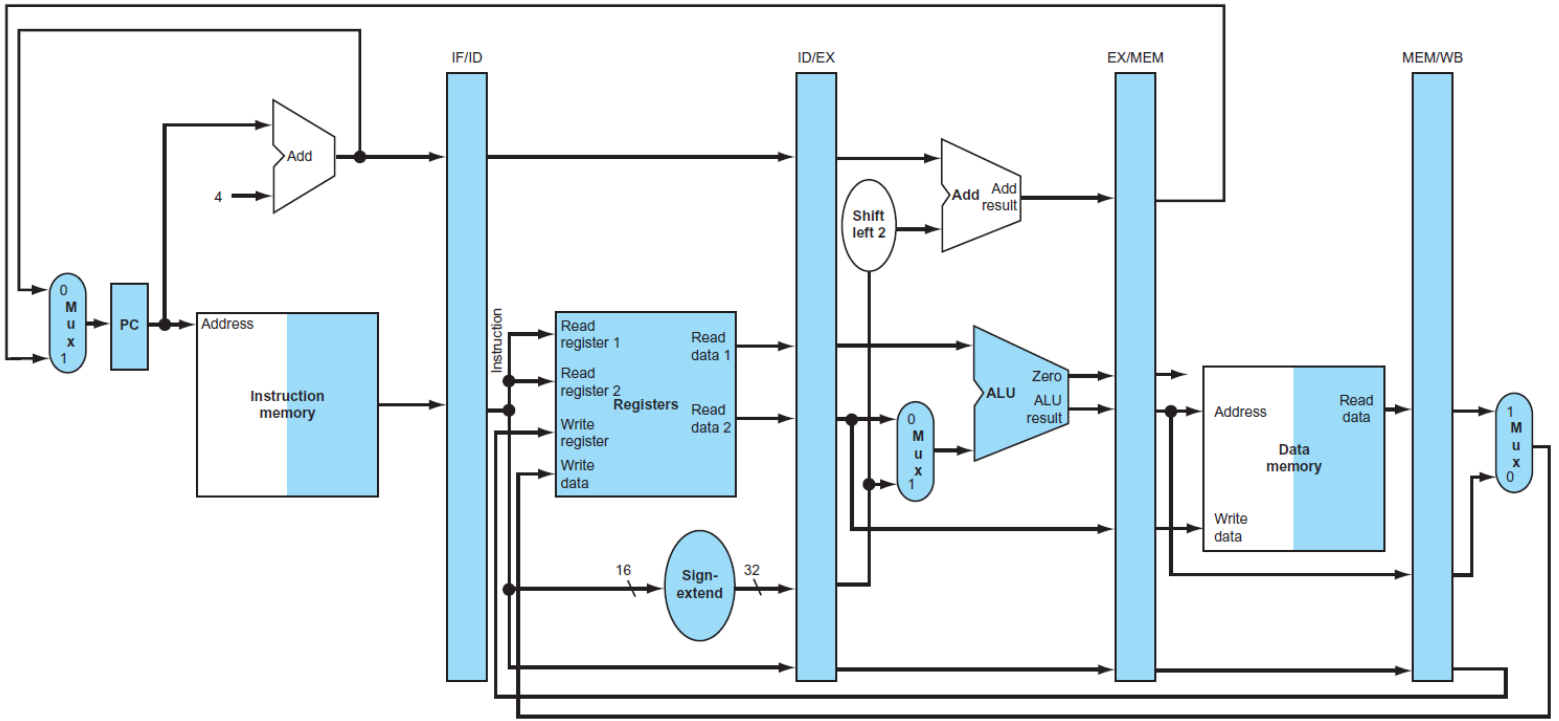
Etapa 5: WB – Escrita nos registradores (sw)



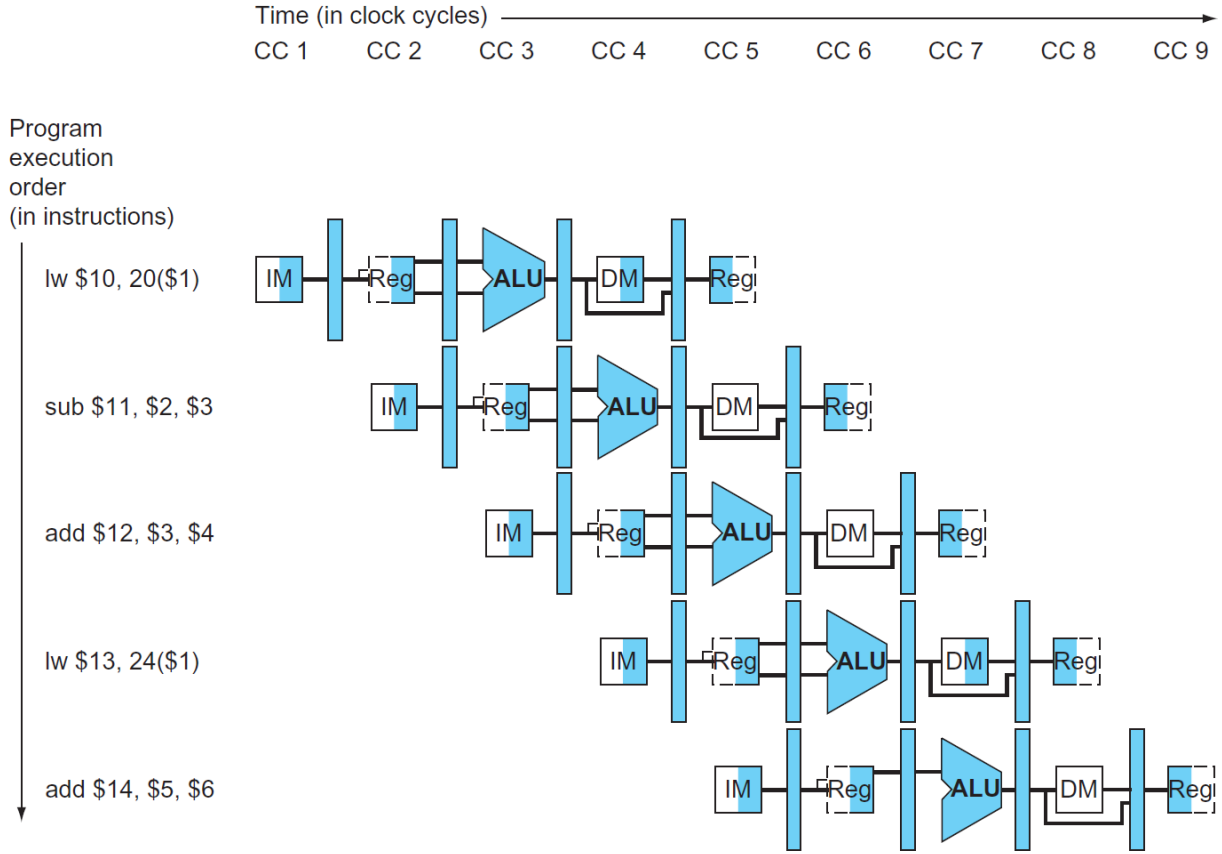
Caminho de dados corrigido – load word



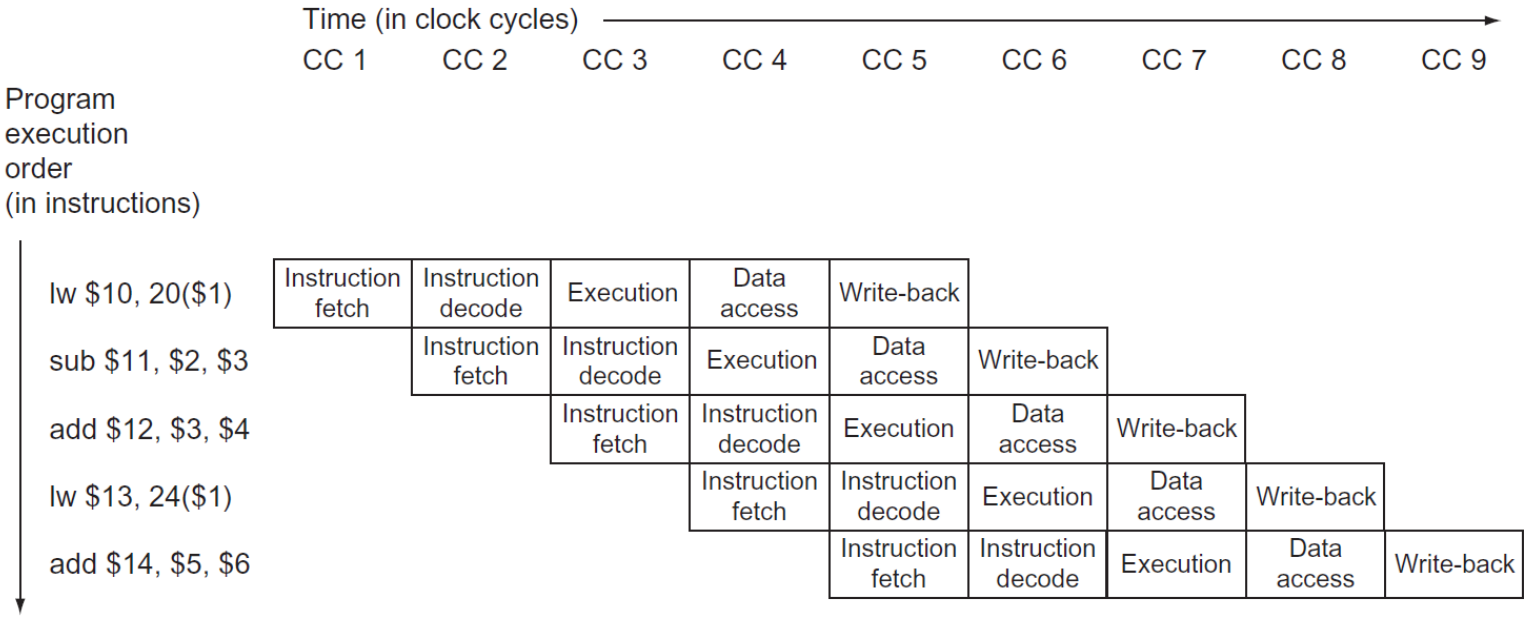
Partes do caminho de dados usado pela instrução lw



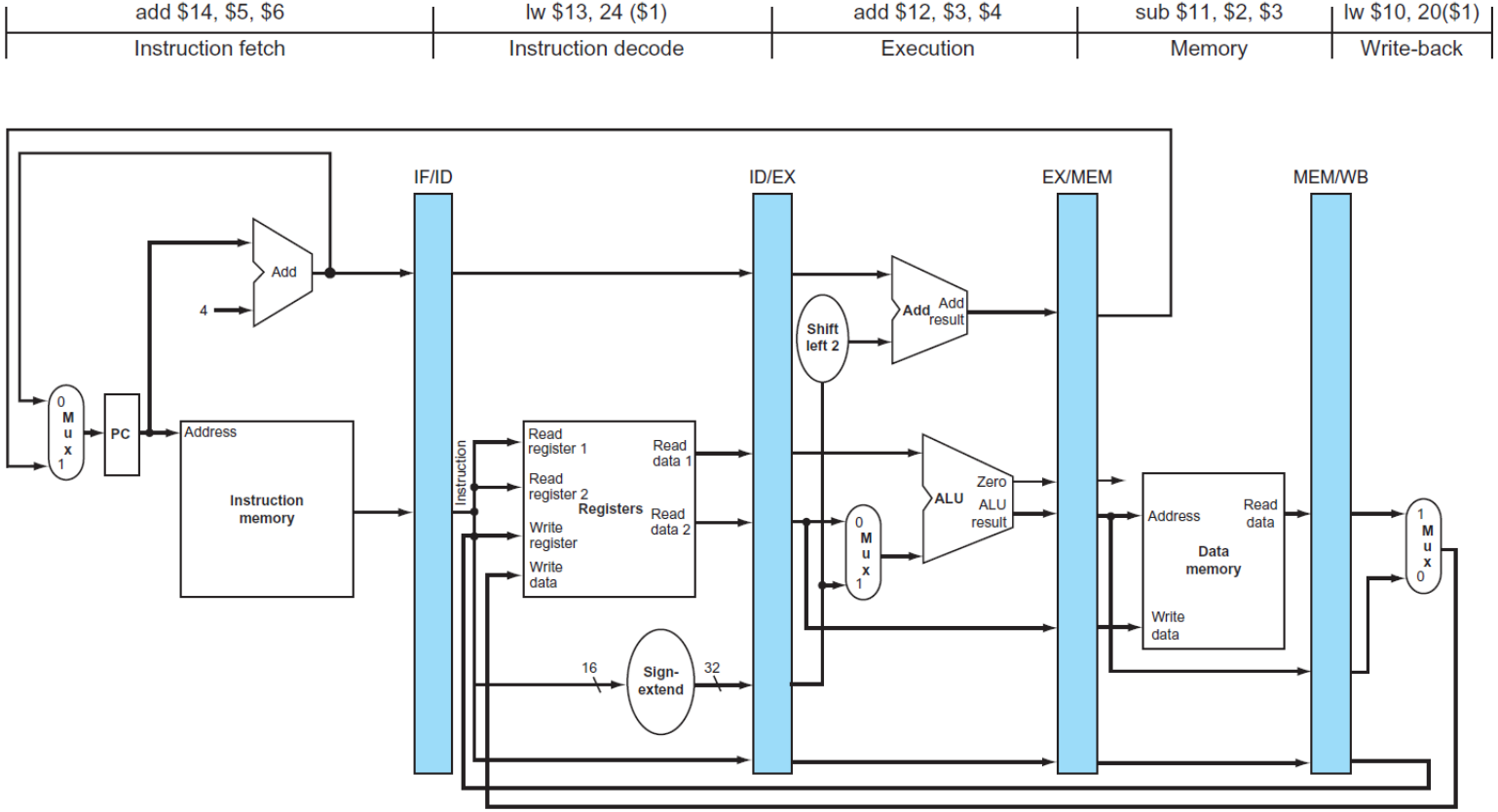
Representando pipelines graficamente



Representando pipelines graficamente

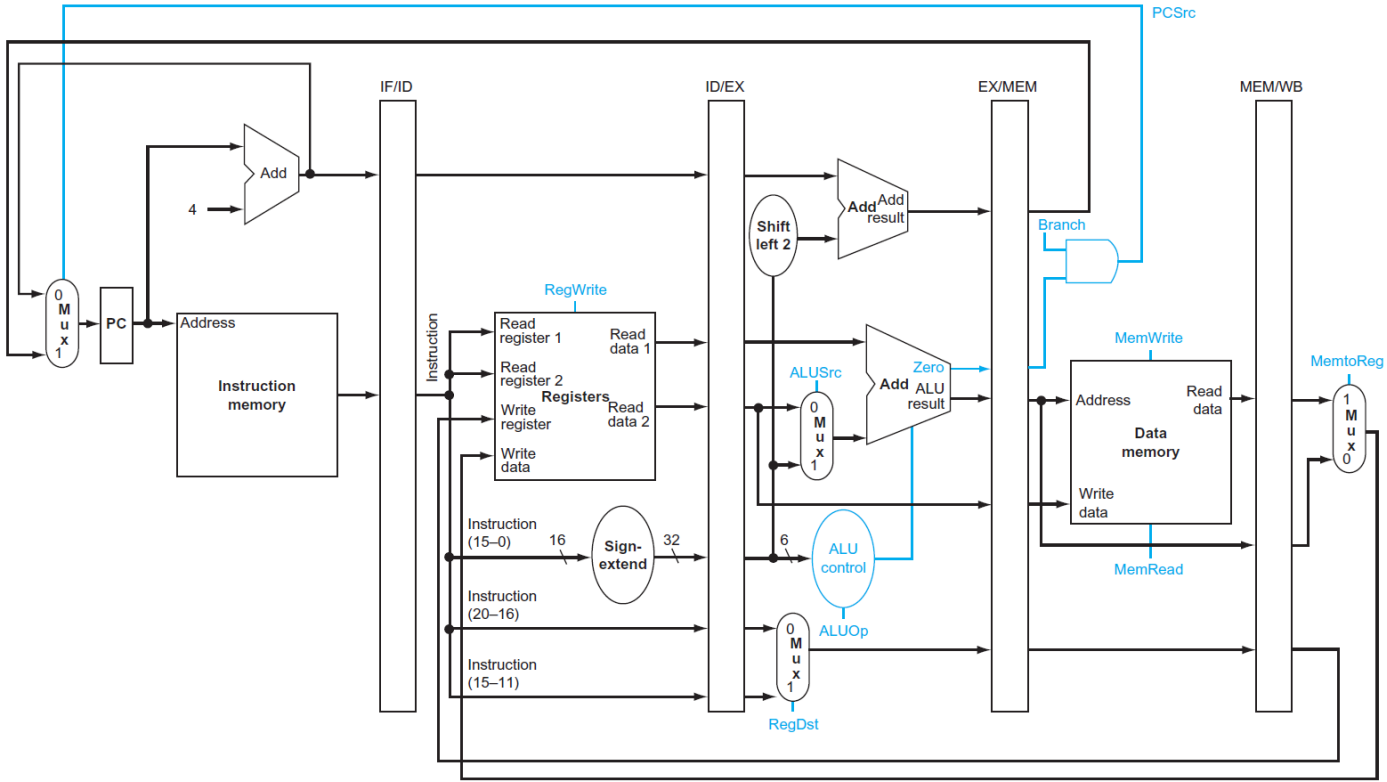


O caminho de dados em um ciclo de clock (CC5)



CONTROLE DE UM PIPELINE

O caminho de dados e sinais de controle



- O caminho de dados com os sinais de controle identificados.

Bits de controle da ULA

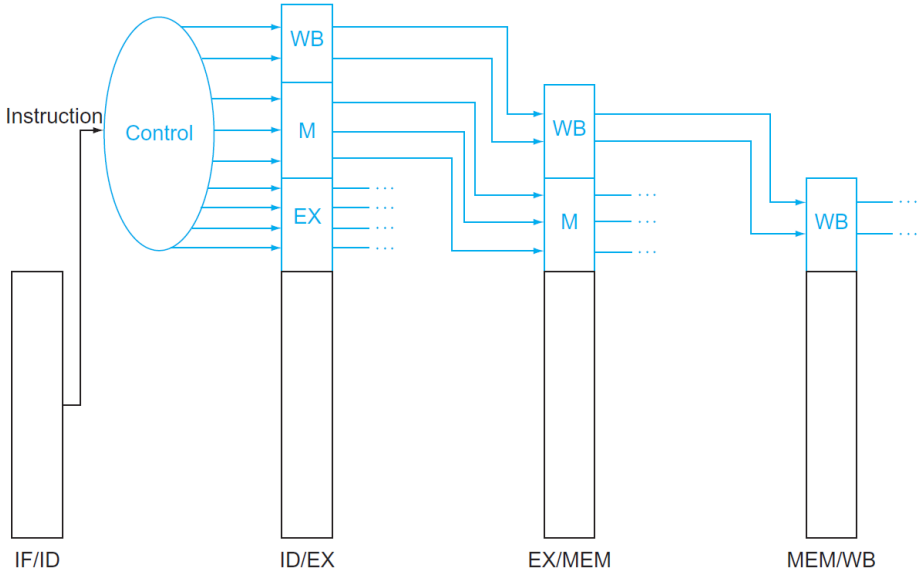
Opcode da instrução	OpALU	Operação da instrução	Campo funct	Ação da ALU desejada	Entrada do controle da ALU
LW	00	load word	XXXXXX	add	0010
SW	00	store word	XXXXXX	add	0010
BEQ	01	branch equal	XXXXXX	subtract	0110
tipo R	10	add	100000	add	0010
tipo R	10	subtract	100010	subtract	0110
tipo R	10	AND	100100	AND	0000
tipo R	10	OR	100101	OR	0001
tipo R	10	set on less than	101010	set on less than	0111

Sinais de controle

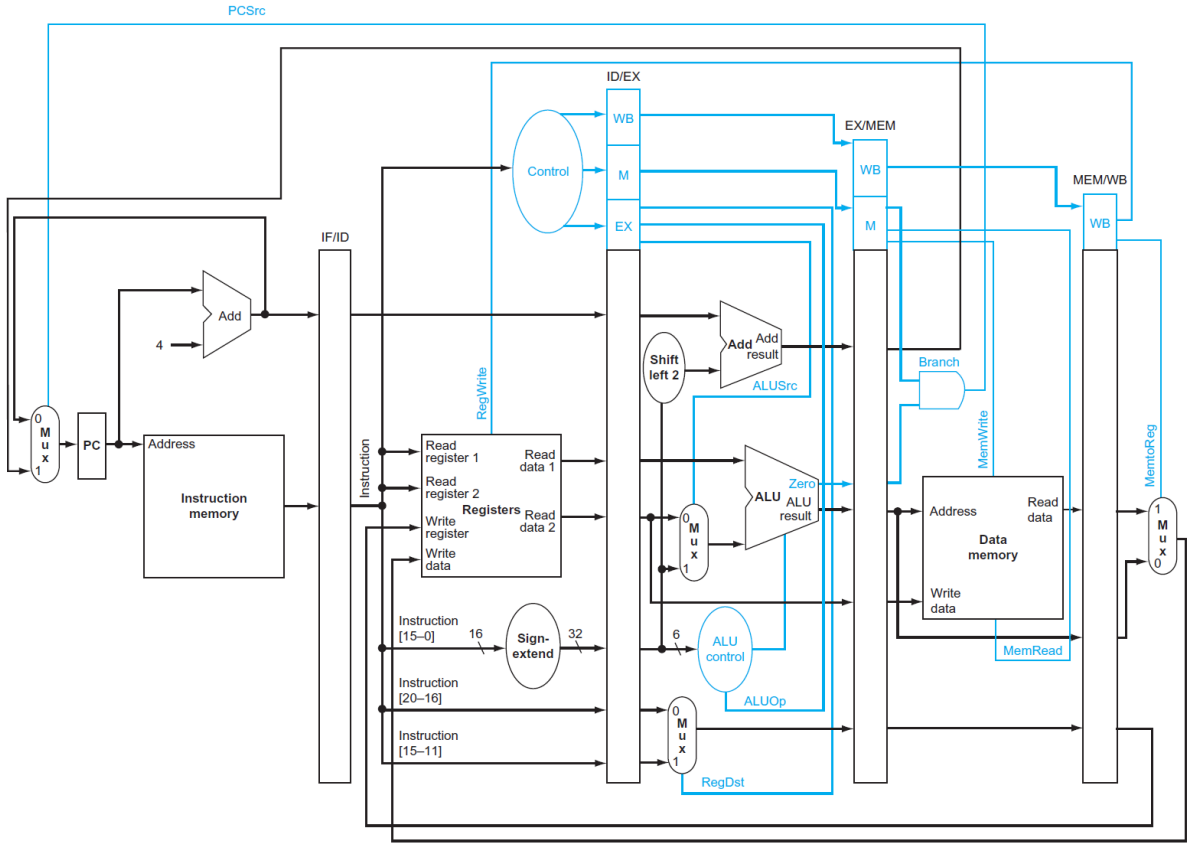
Nome do sinal	Efeito quando inativo	Efeito quando ativo
RegDst	O número do registrador destino para a entrada Registrador para escrita vem do campo rt (bits 20:16)	O número do registrador destino para a entrada ‘Registrador para escrita’ vem do campo rd (bits 15:11)
EscreveReg	Nenhum.	O registrador na entrada ‘Registrador para escrita’ é escrito como o valor da entrada ‘Dados para escrita’.
OrigALU	O segundo operando da ULA vem da segunda saída do banco de registradores (Dados de leitura 2).	O segundo operando da ULA consiste nos 16 bits mais baixos da instrução com sinal estendido.
OrigPC	O PC é substituído pela saída do somador que calcula o valor de PC + 4.	O PC é substituído pela saída do somador que calcula o destino do desvio.
LerMem	Nenhum.	O conteúdo da memória de dados designado pela entrada Endereço é colocado na saída ‘Dados da leitura’.
EscreveMem	Nenhum.	O conteúdo da memória de dados designado pela entrada Endereço é substituído pelo valor na entrada ‘Dados para escrita’.
MemParaReg	O valor enviado para a entrada ‘Dados para escrita’ do banco de registradores vem da ULA.	O valor enviado a entrada ‘Dados para escrita’ do banco de registradores vem da memória de dados.

Bits de controle da ULA

Instrução	Linhas de controle do estágio de execução/cálculo de endereço				Linhas de controle do estágio de acesso à memória			Linhas de controle do estágio de escrita do resultado	
	RegDst	OpALU1	OpALU0	OrigALU	Branch	LeMem	EscreveMem	EscreveReg	MemparaReg
Formato R	1	1	0	0	0	0	0	1	0
lw	0	0	0	1	0	1	0	1	1
sw	X	0	0	1	0	0	1	0	X
beq	X	0	1	0	1	0	0	0	X



O caminho de dados e sinais de controle



- O caminho de dados com os sinais de controle conectados à unidade de controle.

Resumo – Unidade de Controle

- Busca de instruções
- Decodificação de instruções e leitura do banco de registradores
- Execução ou calculo de endereço
  - RegDst, OpALU, OrigALU
- Acesso a memória
  - EscreveMem, LeMem, Branch
- Escrita de resultado
  - MemparaReg, EscreveReg

Bibliografia

- PATTERSON, D. A.; HENNESSY, J. L. **Organização e Arquitetura de computadores: a interface hardware/software**. 3. ed. Rio de Janeiro: Elsevier, 2005.
  - CAPÍTULO 5
- Notas de aula da Profa. Mary Jane Irwin
  - CSE 331 Computer Organization and Design
  - http://www.cse.psu.edu/research/mdl/mji/mjicourses

FIM